BOOK TWO

Backend Complete Guide

The Express and MongoDB API, explained file by file

This is the most detailed document in the set. It teaches the backend rather than just listing it: what each piece does, **why it exists**, and what would go wrong without it.

All paths are relative to `RuWork_backend-master/`.

PROJECT **RUWORK** STATUS **PHASES 1–10 COMPLETE** SCOPE **DOCUMENTATION ONLY**

CONTENTS

§ 1

Backend folder structure

```
RuWork_backend-master/
├── index.js          ← the entry point: builds the app, connects the DB, starts listening
├── models/           ← Mongoose schemas: the SHAPE and RULES of stored data
├── controllers/      ← request handlers: validate input, orchestrate, send a response
├── routes/           ← URL → middleware chain → controller wiring
├── middlewears/      ← reusable request-pipeline steps (auth, security, errors)
├── utils/            ← shared logic with no knowledge of req/res
├── scripts/          ← one-off command-line tools (create admin, seed demo data)
└── tests/            ← automated tests
```

Why separate these at all?

You *could* write the entire backend in one enormous `index.js`. It would work — for about a week. The separation exists for four concrete reasons:

1. Each file has one job, so bugs have one address. If job filtering is broken, you open `controllers/jobController.js`. If a job is being saved with bad data, you open `models/job.js`. You never scroll through 4,000 unrelated lines.

2. Logic gets reused instead of copy-pasted. `utils/account.js` → `normalizeEmail()` is used by student registration, student login, provider registration, provider login, admin login, and password reset. If it lived inside one controller, the other five would each have their own slightly different copy — and one of them would eventually forget to lowercase, creating a security hole where `Student@ruh.ac.lk` and `student@ruh.ac.lk` become two different accounts.

3. Security rules apply automatically instead of by memory. Because authorization lives in `middlewears/authMiddleware.js` and is attached in `routes/`, adding a new admin endpoint automatically inherits the admin guard. If every controller had to remember to check permissions itself, one forgotten check would be a total authorization bypass.

4. Things can be tested in isolation. `utils/` functions are pure logic with no `req`/`res`, so tests can call them directly. That is why the suite runs in about one second with no database.

The layer rule

```
routes/       knows about → middleware + controllers
controllers/  knows about → utils + models        (and req/res)
utils/        knows about → models (sometimes)     (never req/res)
models/       knows about → nothing but Mongoose
```

Notice utilities never touch `req` or `res`. That is what makes them trivially testable.

> **Note on the spelling:** the directory really is `middlewears/`, not `middlewares/`. It came from the original supplied backend and was deliberately kept so the diff stayed small. Don't "fix" it — every import in the project points at it.

§ 2

Server startup — index.js

`index.js` runs top to bottom when you run `npm start`. Here is what happens, in order.

2.1 Imports and environment loading

```js
import "dotenv/config";
import express from "express";
import mongoose from "mongoose";
// … routers, controllers, middleware, utils
```

`import "dotenv/config"` is **first on purpose**. It reads the `.env` file and copies every `KEY=value` into `process.env`. Any module imported afterwards can read `process.env.JWT_SECRET`. If this line came later, some module might read a secret before it existed and get `undefined`.

2.2 Building the Express app

```js
const app = express();
app.disable("x-powered-by");
app.set("trust proxy", getTrustProxySetting());
```

- `express()` creates the application object.
- `app.disable("x-powered-by")` removes the `X-Powered-By: Express` response header. **Why:** it tells attackers exactly what software and therefore which known exploits to try. Free information for them, zero benefit to us.
- `app.set("trust proxy", …)` tells Express whether to believe the `X-Forwarded-For` header. **Why it matters:** behind a load balancer every request appears to come from the balancer's IP. Rate limiting would then see *all* users as one client and lock everyone out together. Configured by `TRUST_PROXY`.

> **Security reason:** never set `trust proxy` to `true` when you are *not* behind a proxy. Any client could then forge `X-Forwarded-For` and rotate their apparent IP to bypass rate limits entirely. That is exactly why it defaults to `false`.

2.3 The global middleware stack — and why order matters

```JS
app.use(securityHeaders());     // 1
app.use(corsPolicy());          // 2
app.use(express.json({ limit: getJsonBodyLimit() }));  // 3
app.use(requireObjectBody);     // 4
app.use(apiRateLimiter);        // 5
```

Every request passes through these in sequence. The order is a deliberate defensive design:

#	MIDDLEWARE	WHY IT MUST BE HERE
1	`securityHeaders()`	Headers must be attached to **every** response, including errors and rejections. First position guarantees no response can escape without them.
2	`corsPolicy()`	Handles the browser's `OPTIONS` preflight and rejects disallowed origins before any work is done.
3	`express.json({ limit })`	Parses the JSON body into `req.body`. The limit (100 kB) is applied *during* parsing — a 2 GB body is rejected while streaming, never buffered into memory.
4	`requireObjectBody`	Runs **after** parsing (it needs `req.body`) but **before** any controller, so a body that parsed into an array or string is rejected once, centrally.
5	`apiRateLimiter`	Last, so throttled requests have already been given security headers and CORS treatment and get a correct, well-formed `429`.

What could go wrong if the order changed? Put `express.json()` before the body-size decision and a single malicious request could exhaust server memory. Put the rate limiter first and a `429` response would ship without security headers. Put CORS after parsing and you'd waste CPU parsing bodies from origins you were about to reject.

2.4 Routes

```JS
app.get("/api/health", getHealth);
app.use("/api/users", userRouter);
app.use("/api/admin", adminRouter);
app.use("/api/jobProviders", JobProviderRouter);
app.use("/api/jobs", jobRouter);
app.use("/api/applications", applicationRouter);
app.use("/api/reviews", reviewRouter);
app.use("/api/messages", messageRouter);
app.use("/api/notifications", notificationRouter);
```

`app.use("/api/users", userRouter)` means *"for any URL starting with `/api/users`, hand the request to `userRouter`, and let it match against the remainder of the path."* So `userRouter.post("/login", …)` serves `POST /api/users/login`. This mounting is what keeps each router file focused on one area.

Health is registered directly rather than in a router because it is a single endpoint with no shared middleware.

2.5 The two terminal handlers

```js
app.use(notFoundHandler);
app.use(errorHandler);
```

These are **last** and their order is mandatory.

- `notFoundHandler` runs only if no route matched — so it returns a JSON `404` instead of Express's default HTML page (an API client parsing JSON should never receive HTML).
- `errorHandler` takes **four** parameters `(error, req, res, next)`. Express uses the parameter count to recognise error handlers. **Three parameters and it silently becomes normal middleware that never runs** — a classic and very confusing bug.

2.6 Startup sequence

```js
async function startServer() {
    assertEnvironment();                              // 1 fail fast
    await mongoose.connect(process.env.MONGODB_URI.trim());  // 2
    logger.info("MongoDB connection established");
    const server = app.listen(getPort(), () => { … }); // 3
    // 4 connection lifecycle listeners
    // 5 graceful shutdown
}
```

1. `assertEnvironment()` first. Configuration is validated *before* anything connects or listens. If `JWT_SECRET` is missing, the server refuses to start rather than starting and then failing on every login. Failing fast at boot is far better than failing mysteriously at 2 a.m. under load.

2. Connect to MongoDB before listening. If the database is unreachable the process exits with a non-zero code instead of accepting traffic it cannot serve.

3. Then listen. Only once configuration and the database are confirmed.

4. Connection lifecycle listeners. `disconnected` / `reconnected` / `error` are logged. The initial connection succeeding does not mean it stays up.

5. Graceful shutdown.

```js
const shutdown = async (signal) => {
    logger.info("Shutting down", { signal });
    server.close();
    await mongoose.connection.close().catch(() => {});
    process.exit(0);
};
process.on("SIGINT", () => shutdown("SIGINT"));
process.on("SIGTERM", () => shutdown("SIGTERM"));
```

Why: `SIGTERM` is what Docker/Kubernetes sends before killing a container. Without this, in-flight requests are cut mid-write and database connections are abandoned. With it, the server stops accepting new connections and closes cleanly.

Finally, `export default app;` exists so tests can import the app without starting a listener.

§ 3

Express fundamentals

3.1 The core objects

`express()` creates the app — the container for all middleware and routes.

`express.Router()` creates a mini-app you can mount under a path prefix. Every file in `routes/` is a Router.

`req` (request) — what the client sent:

PROPERTY	CONTAINS	RUWORK EXAMPLE
`req.params`	Values from `:placeholders` in the path	`GET /api/jobs/abc123` → `req.params.id` `===` `"abc123"`
`req.query`	Values after `?`	`GET /api/jobs?page=2&category=Tutoring` → `req.query.page` `===` `"2"`
`req.body`	The parsed JSON body	`{ email, password }` on login
`req.get("Header")`	A request header	`req.get("Authorization")` → `"Bearer eyJ…"`
`req.user`	**Added by RuWork**, not Express — the verified JWT claims	set by `authenticateToken()`
`req.studentAccount`	**Added by RuWork** — the live Student document	set by `requireEligibleRuhunaStudent()`

> ⚠ **`req.params` and `req.query` values are always strings.** `req.query.page` is `"2"`, not `2`. This is why `utils/admin.js` → `adminPagination()` converts and validates rather than trusting the type.

`res` (response) — how you reply:

```js
return res.status(201).json({ message: "Job published successfully", job: … });
```

`res.status(code)` sets the HTTP status and returns `res` so you can chain. `res.json(obj)` serialises to JSON, sets `Content-Type: application/json`, and sends. **A response can only be sent once** — hence the `return` in front of every one in RuWork. Forgetting it causes the dreaded *"Cannot set headers after they are sent"*.

`next` — hands control to the next middleware. Call `next()` to continue, or `next(error)` to jump straight to `errorHandler`.

3.2 Middleware vs controller

A **middleware** sits in the middle of the pipeline. It either passes the request along (`next()`) or ends it (`res.status(…).json(…)`).

```JS
// middlewears/authMiddleware.js — simplified
export function authenticateToken(req, res, next) {
    const [scheme, token] = (req.get("Authorization") || "").split(" ");
    if (scheme !== "Bearer" || !token) {
        return res.status(401).json({ error: "Authentication is required" });  // STOP
    }
    try {
        req.user = jwt.verify(token, process.env.JWT_SECRET);
        return next();                                                          // CONTINUE
    } catch {
        return res.status(401).json({ error: "Invalid or expired authentication token" });
    }
}
```

A **controller** is the final handler that produces the answer. By the time it runs, all guards have passed, so it can focus on business logic.

```JS
routes/userRouter.js
userRouter.get("/profile",
    authenticateToken,              // middleware — is the token valid?
    isStudent,                      // middleware — is the role "student"?
    requireEligibleRuhunaStudent,   // middleware — is the live account eligible?
    getMyProfile                    // controller — send the profile
);
```

3.3 HTTP methods used in RuWork

METHOD	MEANING	RUWORK EXAMPLE
`GET`	Read. Never changes data.	`GET /api/jobs`
`POST`	Create something new.	`POST /api/jobs`
`PATCH`	Partially update an existing thing.	`PATCH /api/jobs/:id`
`DELETE`	Remove.	`DELETE /api/jobs/:id` — but note this **archives**, it does not erase

RuWork uses `PATCH` rather than `PUT` because clients send only the fields they want changed, never the whole document.

3.4 Status codes and what they mean here

CODE	MEANING	RUWORK EXAMPLE
200 OK	Success with a body	Login succeeded; job list returned
201 Created	A new record exists	Registration, job creation, application, review, message
400 Bad Request	The input was invalid	Password too weak; note under 20 characters; malformed JSON
401 Unauthorized	*"I don't know who you are."*	Missing/expired/invalid token; wrong password; revoked token
403 Forbidden	*"I know who you are, and you may not do this."*	Student token on an admin route; unapproved provider posting a job
404 Not Found	The resource does not exist (or you may not see it)	Unknown job id; malformed ObjectId
409 Conflict	Valid request, but it clashes with current state	Duplicate email; applying twice; approving an already-approved registration
413 Payload Too Large	Body exceeded the limit	JSON body over 100 kB
429 Too Many Requests	Rate limit hit	11th failed login inside 15 minutes
500 Internal Server Error	An unexpected bug	Anything unhandled — always returns a generic message
503 Service Unavailable	Temporarily broken, try later	Health check when MongoDB is down; verification email could not be sent

3.5 401 vs 403 — the distinction that confuses everyone

They sound the same. They are not.

- **401 = authentication problem.** The server does not know who you are. *Fix: log in.*
- **403 = authorization problem.** The server knows exactly who you are and is refusing anyway. *Fix: nothing you can do; you lack permission.*

In RuWork:

```
No Authorization header                      → 401   (authenticateToken)
Expired or tampered token                    → 401   (authenticateToken)
Token whose tv claim no longer matches       → 401   (TOKEN_REVOKED)
Valid Student token used on /api/admin/*     → 403   (isAdmin)
Valid Student token, but account suspended   → 403   (STUDENT_NOT_ELIGIBLE)
Provider editing a job they do not own       → 403
```

Why it matters practically: the frontend uses this distinction. `services/api.js` clears the session on a `401` (your credentials are dead — sign in again) but leaves it alone on a `403` (you are still logged in; you just can't do that).

§ 4

MongoDB and Mongoose

4.1 MongoDB concepts

MongoDB is a **document database**. Compared to SQL:

SQL TERM	MONGODB TERM	RUWORK EXAMPLE
Table	**Collection**	`jobs`, `users`, `applications`
Row	**Document**	one job posting
Column	**Field**	`jobTitle`, `hourlyRate`
Primary key	**`_id`** (an ObjectId)	`507f1f77bcf86cd799439011`

A document is essentially JSON. Arrays and nested objects are natural — `requiredSkills: ["Research", "Data Entry"]` is a plain array field, no join table required.

ObjectId is MongoDB's 12-byte unique id, shown as a 24-character hex string. It embeds a creation timestamp, so sorting by `_id` roughly sorts by age. RuWork validates them with `mongoose.isValidObjectId(value)` before querying — passing a malformed id straight to a query throws a `CastError`, which is why controllers check first and return a clean `404`.

4.2 Why Mongoose

MongoDB by itself will happily store `{ jobTitle: 12345 }` or a job with no price. Mongoose adds a **schema**: types, `required`, `enum`, `min`/`max`, `default`, custom validators, indexes, and timestamps. It is the layer that makes the data trustworthy.

4.3 Schema options used in RuWork

```js
jobTitle: { type: String, required: true, trim: true, maxlength: 120 }
```

OPTION	EFFECT	WHY RUWORK USES IT
`type`	Declares the data type	Prevents a number being stored where text belongs
`required`	Rejects a missing value	A job with no title is meaningless
`default`	Fills in a value when absent	`accountStatus: "pending"` — new accounts are never accidentally approved
`enum`	Restricts to a fixed list	`status: ["draft","open","closed"]` — no invented states
`trim`	Strips surrounding whitespace	`" Matara "` and `"Matara"` become the same value
`lowercase`	Forces lower case	Emails, so `A@x.lk` and `a@x.lk` cannot become two accounts
`unique`	Creates a unique index	One account per email; one review per application
`immutable`	Value can never change after creation	Ownership and audit fields
`select: false`	Excluded from query results **by default**	Token hashes and `priceAmount`
`min` / `maxlength`	Bounds	Prices > 0; comments ≤ 1000 chars
`validate`	Custom rule	Rating must be a whole number

`immutable` deserves emphasis. On `Application`, `jobId`, `studentId`, and `jobProviderId` are immutable. Even if a bug tried to reassign an application to a different student, Mongoose silently refuses. It is a last line of defence behind the controller checks.

`select: false` is a privacy default. `emailVerificationTokenHash` is `select: false`, so an ordinary `User.findById()` never loads it — it cannot leak into a response by accident. Code that genuinely needs it must ask explicitly:

```js
Model.findOne(query).select("+emailVerificationTokenHash +emailVerificationExpiresAt")
```

`timestamps: true` adds `createdAt` and `updatedAt` automatically. `AdminAudit` uses `{ createdAt: true, updatedAt: false }` — an audit record is never updated, so an `updatedAt` field would be a lie.

4.4 Query methods you will meet

METHOD	RETURNS	NOTES
`Model.find(filter)`	Array of documents	Chain `.sort() .skip() .limit()`
`Model.findOne(filter)`	One document or `null`	Used for login lookups
`Model.findById(id)`	One document or `null`	Used by every guard
`Model.countDocuments(filter)`	A number	Pagination totals, dashboard stats
`Model.exists(filter)`	Truthy or `null`	Cheap "does this exist?"
`doc.save()`	Persists changes, runs validation	The main write path
`Model.updateMany(filter, update)`	Bulk update	Provider suspension stamping their jobs
`Model.deleteOne(filter)`	Removes a document	Rare — only reviews and demo cleanup
`Model.aggregate(pipeline)`	Computed results	Rating averages, dashboard grouping

`.lean()` returns plain JavaScript objects instead of full Mongoose documents. They are lighter and faster but have no `.save()`. RuWork uses `.lean()` for read-only lists and full documents when it needs to modify and save.

`.populate()` follows a reference and swaps the id for the actual document:

```js
Job.find(filter).populate({ path: "jobProviderId", select: "companyName industry companyWebsite" })
```

Without populate you get `jobProviderId: "507f…"`. With it you get the company object. The `select` is deliberate — it fetches *only* those three fields, so the provider's email and password hash cannot leak into a public job listing.

Aggregation runs a pipeline of stages in the database:

```js
// utils/ratingAggregates.js
Review.aggregate([
    { $match: { jobId, moderationStatus: { $ne: "hidden" } } },
    { $group: { _id: null, averageRating: { $avg: "$rating" }, reviewCount: { $sum: 1 } } }
]);
```

`$match` filters, `$group` computes the average and count. **Why aggregate instead of fetching every review and averaging in JavaScript?** A job with 10,000 reviews would mean transferring 10,000 documents over the network. Aggregation returns one small result and the database does the arithmetic.

4.5 Indexes

An index is a sorted lookup structure. Without one, MongoDB scans every document (a "collection scan") — fine with 50 jobs, catastrophic with 500,000.

```js
jobSchema.index({ moderationStatus: 1, providerSuspendedAt: 1, archivedAt: 1, status: 1, createdAt: -1 });
```

`1` is ascending, `-1` descending. This **compound** index exactly matches the public browse query, so the most-used endpoint stays fast.

Two special ones:

```js
applicationSchema.index({ jobId: 1, studentId: 1 }, { unique: true });
```

> **Security/correctness reason:** this is what actually prevents duplicate applications. A controller check ("has this student already applied?") has a race window — two simultaneous requests can both read "no" and both insert. The unique index makes the *database* reject the second write with error code `11000`, which the controller converts to a clean `409`. **The check is UX; the index is the guarantee.**

```js
jobSchema.index({ jobTitle: "text", jobDescription: "text", companyName: "text", requiredSkills: "text"
});
```

A **text index** powers `$text` keyword search across those four fields.

4.6 The models

`MODELS/USER.JS` — STUDENT

Represents a University of Ruhuna student.

Personal: `firstName`, `lastName`, `email`, `phoneNumber`, `dateOfBirth`, `gender` (enum: `Male` / `Female` / `Prefer not to say`).

Academic: `university` (enum containing only `University of Ruhuna`, `immutable`), `faculty`, `fieldOfStudy` (required), `yearOfStudy` (required).

The email field is the most defended in the schema:

```js
email: {
    type: String, required: true, unique: true, trim: true, lowercase: true,
    set: normalizeEmail,
    validate: { validator: isAllowedStudentEmail,
                message: "Email must use the official University of Ruhuna domain" }
}
```

Four layers: trimmed, lowercased, run through `normalizeEmail`, then validated for the exact `ruh.ac.lk` domain — plus a unique index. **Why so much?** Without normalisation `" Student@RUH.AC.LK "` and `"student@ruh.ac.lk"` are different strings, so the unique index would allow both, creating two accounts for one person and an account-takeover vector at login.

Lifecycle: `isEmailVerified` (default `false`), `accountStatus` (`pending`/`approved`/`rejected`, default `pending`), `rejectionReason`, `reviewedAt`, `reviewedBy` (`select: false`).

Verification internals (all `select: false`): `emailVerificationTokenHash`, `emailVerificationExpiresAt`, `verificationEmailSentAt`.

Moderation (Phase 9): `moderationStatus` (`active`/`suspended`), `moderationReason`, `moderatedAt`, `moderatedBy`.

Credentials/session (Phase 10): `password` (the bcrypt hash), `passwordResetTokenHash`, `passwordResetExpiresAt`, `passwordResetRequestedAt` (all `select: false`), `passwordChangedAt`, `tokenVersion` (default `0`).

`role`: enum containing only `student`, `immutable`.

> **Security reason for `immutable` on `role` and `university`:** even if a bug passed the whole request body into the model, a client could not promote themselves. Combined with the controller building the document field-by-field, that is two independent defences against privilege escalation.

Index: `{ accountStatus: 1, moderationStatus: 1, createdAt: -1 }` — serves the Admin student listing.

MODELS/JOBPROVIDER.JS — JOB PROVIDER

Company: `companyName`, `companyEmail` (canonical, normalised, unique — *not* domain-restricted), `phoneNumber`, `companyAddress`, `companySize`, `industry`, `companyWebsite` (optional), `companyDescription`.

Contact person: `firstName`, `lastName`.

Same lifecycle, verification, moderation, and credential blocks as `User`.

Rating summary: `averageRating` (nullable) and `reviewCount` (default `0`) — denormalised across *all* the provider's jobs.

`role`: enum containing only `Job_Provider`, immutable.

> **Why is the field `companyEmail` and not `email`?** *Historical, and it caused a real Phase 1 bug: the model stored `companyEmail` while login queried `email`, so provider login could never succeed. The fix standardised on `companyEmail` everywhere. This is why `controllers/passwordController.js` carries an `emailField` per account type instead of assuming `email`.*

MODELS/ADMIN.JS — ADMIN

Deliberately minimal: `firstName`, `lastName`, `email` (unique, normalised), `password`, `passwordChangedAt`, `tokenVersion`, `role` (enum `admin`, immutable).

No `accountStatus`, no verification fields, no moderation fields. **Why:** Admins are created by a human running a script on the server; there is no self-service flow to approve, verify, or suspend.

MODELS/JOB.JS — JOB

Ownership: `jobProviderId` (required, `immutable`) and `companyName` (a synchronised copy of the current provider name, kept so listings need no join).

Content: `jobTitle` (≤120), `jobDescription` (≤2000), `category` (enum), `scope` (≤1000), `location`, `workingHours`, `requiredSkills` (1–10 unique tags, each ≤50 chars, normalised by `normalizeSkills`), `suitableFor` (enum), `applicationDeadline`.

`JOB_CATEGORIES` (from `utils/job.js`): `Delivery`, `Buy and Sell`, `Tutoring`, `Event Support`, `Data Entry`, `Content Creation`, `Other`. `JOB_SUITABLE_YEARS`: `Any Year`, `1st Year`, `2nd Year`, `3rd Year`, `4th Year`, `Final Year`.

Pricing: `budgetType` (`hourly` | `fixed`), `hourlyRate`, `budget`, `priceAmount` (`select: false`), `currency` (enum `LKR`, immutable).

A `pre("validate")` hook enforces the pairing:

```js
jobSchema.pre("validate", function validatePricing() {
    if (this.budgetType === "hourly") {
        if (!Number.isFinite(this.hourlyRate) || this.hourlyRate <= 0) {
            this.invalidate("hourlyRate", "Hourly jobs require an hourly rate greater than zero");
        }
        this.budget = undefined;          // clear the irrelevant one
        this.priceAmount = this.hourlyRate;
    } else if (this.budgetType === "fixed") { /* mirror image */ }
});
```

Why `priceAmount`? Sorting and range-filtering by price must work across both pricing models in one query. Without a unified field you would need two branches in every filter. It is `select: false` because it is an internal mechanism, not something to show a user.

Lifecycle: `status` (`draft` | `open` | `closed`), `archivedAt` (default `null`).

Ratings: `averageRating` (nullable, 1–5), `reviewCount`.

Moderation: `moderationStatus` (`visible` | `hidden`), `moderationReason`, `moderatedAt`, `moderatedBy`, plus `providerSuspendedAt`.

> **Why is `providerSuspendedAt` separate from `moderationStatus`?** *They are two independent reasons a job can be invisible. If suspending a provider set every job to `hidden`, then restoring the provider would have to guess which jobs were already individually hidden by an Admin — and would wrongly un-hide them. Two separate fields keep the two decisions independent and perfectly reversible.*

MODELS/APPLICATION.JS — APPLICATION

The link between a Student and a Job.

Immutable references: `jobId`, `studentId`, `jobProviderId`.

Content: `applicationNote` (required, 20–1000 characters).

> ⚠ The field is `applicationNote`. `scripts/seedDemo.js` currently uses `studentNote` — see §24.

Status: enum `APPLICATION_STATUSES` = `pending_review`, `in_progress`, `completed`, `declined`, `withdrawn`, `cancelled` (default `pending_review`).

Pricing snapshot: `budgetType` (immutable), `originalHourlyRate` / `originalBudget` (immutable), `approvedHourlyRate` / `approvedBudget` (mutable), `currency`.

> **Why snapshot the original price?** *The provider may later edit the job's rate. Without a snapshot, historical applications would appear to have been made at today's price. The immutable snapshot preserves what was actually on offer when the student applied. The `approved*` fields are separate because the provider may negotiate a different figure on acceptance.*

Reasons and timestamps: `declineReason`, `cancellationReason`, `appliedAt` (immutable), `acceptedAt`, `declinedAt`, `withdrawnAt`, `cancelledAt`, `completedAt`.

A `pre("validate")` hook clears the irrelevant price pair and requires an approved price once status is `in_progress`, `completed`, or `cancelled`.

Indexes: the unique `{ jobId, studentId }` plus three compound indexes for the student list, the job's applicants, and the provider's list.

MODELS/REVIEW.JS — REVIEW

`applicationId` (immutable **and `unique`**), `jobId`, `studentId`, `jobProviderId` (all immutable), `rating` (1–5, must be a whole number), `comment` (≤1000, default `""`), moderation block.

> **Why is `applicationId` unique?** *It is the database-level guarantee of "one active review per completed engagement". Like duplicate applications, the controller check is UX and the unique index is the actual rule. A duplicate insert returns `11000`, which `reviewController` maps to `409 REVIEW_ALREADY_EXISTS`.*

MODELS/MESSAGE.JS — MESSAGE

There is **no Conversation model**. A conversation is simply "all messages sharing an `applicationId`", ordered by time.

`senderType` / `senderId`, `receiverType` / `receiverId` (participant type enum `student` | `jobProvider`), `jobId`, `applicationId` — all immutable. `content` (1–2000 chars), `sharedContactNumber` (default `null`, immutable), `isRead`, `readAt`.

> **Why store the participant *type* alongside the id?** *Students and Providers live in two different collections, so an ObjectId alone is ambiguous — it could point at either. The discriminator makes every message unambiguous without merging the collections.*

MODELS/NOTIFICATION.JS — NOTIFICATION

`recipientType` / `recipientId`, `type` (one of seven), `message` (≤500), `relatedJobId` / `relatedApplicationId` / `relatedMessageId`, `isRead`, `readAt`. Everything except the read state is immutable.

The seven types: `NEW_APPLICATION`, `APPLICATION_ACCEPTED`, `APPLICATION_DECLINED`, `APPLICATION_WITHDRAWN`, `APPLICATION_CANCELLED`, `APPLICATION_COMPLETED`, `NEW_MESSAGE`.

MODELS/ADMINAUDIT.JS — ADMINAUDIT

Append-only record of one Admin decision.

`adminId`, `action` (enum of 12), `entityType` (enum of 6), `entityId`, `metadata` (Mixed, validated to ≤1500 serialised characters) — **every field immutable** — plus `createdAt` only.

Indexes: `{ createdAt: -1 }`, `{ adminId: 1, createdAt: -1 }`, `{ entityType: 1, entityId: 1, createdAt: -1 }`.

MODELS/PLATFORMSETTING.JS — PLATFORMSETTING

A **singleton**: `singletonKey` (enum containing only `"platform"`, `unique`, `immutable`) guarantees at most one document ever exists. Holds exactly three booleans — `studentRegistrationOpen`, `providerRegistrationOpen`, `jobPostingOpen` — plus `updatedBy` and timestamps. See §17.

§ 5

Authentication system

5.1 The full journey

```
REGISTER  →  password hashed with bcrypt, account saved as pending + unverified
   ↓
VERIFY    →  user clicks emailed link, isEmailVerified = true
   ↓
APPROVE   →  Admin sets accountStatus = approved
   ↓
LOGIN     →  password compared, all gates re-checked, JWT issued
   ↓
STORE     →  frontend keeps the JWT in sessionStorage
   ↓
REQUEST   →  Authorization: Bearer <token> on every call
   ↓
VERIFY    →  authenticateToken() checks the signature → req.user
   ↓
RE-READ   →  guard loads the account FROM MONGODB again
   ↓
AUTHORIZE →  role, eligibility, moderation, tokenVersion all re-checked
   ↓
CONTROLLER runs
```

5.2 bcrypt and why passwords are never stored

```js
const hashedPassword = await bcrypt.hash(dataU.password, 10);   // registration
const ok = await bcrypt.compare(submitted, user.password);      // login
```

A **hash** is a one-way transformation. `bcrypt.hash("MyPass123", 10)` produces something like `$2b$10$N9qo8uLOickgx2ZMRZo…`, and there is no function that turns it back.

The `10` is the **cost factor**: bcrypt runs 2^{10} internal rounds, taking roughly 100 ms. That is imperceptible for one login and devastating for an attacker trying billions of guesses. bcrypt also **salts** automatically — a random value mixed into each hash — so two users with the same password get different hashes, defeating precomputed rainbow tables.

What goes wrong without it: a single database leak exposes every password in plain text. Because people reuse passwords, that compromises their email and bank accounts too. This is the single highest-impact security decision in the whole project.

> **Security reason:** RuWork never logs, returns, or emails a password. Login compares hashes; reset *replaces* the hash. Nobody — including an Admin — can read a user's password.

5.3 JWT

A JSON Web Token is three base64url segments separated by dots: `header.payload.signature`.

RuWork's payload, built in `utils/account.js` → `createAccessToken()`:

```js
jwt.sign({
    sub: account._id.toString(),   // subject — WHO
    firstName, lastName, email,
    role: account.role,            // student | Job_Provider | admin
    tv: Number(account.tokenVersion || 0)   // revocation counter
}, jwtSecret, { expiresIn: process.env.JWT_EXPIRES_IN || "1d" });
```

The payload is only base64-encoded, not encrypted. Anyone can decode and read it — which is exactly what the frontend does in `utils/token.js` to know which menu to draw. Never put a secret in a JWT.

The signature is what matters. It is an HMAC of the header and payload using `JWT_SECRET`. Change one character of the payload and the signature no longer matches. Because only the server knows the secret, only the server can mint a valid token.

> **Security reason — why frontend role values cannot be trusted:** the browser could set `sessionStorage` to `{"user":{"role":"admin"}}` and the UI would draw admin menus. It would achieve nothing. Every API call carries the *token*, and `authenticateToken()` verifies the signature server-side. A forged role never survives verification. **The frontend role is cosmetic; the token is authoritative.**

`expiresIn` bounds the damage from a stolen token. Without expiry a leaked token works forever.

5.4 `tokenVersion` / the `tv` claim

JWTs are *stateless* — the server does not store issued tokens, so it cannot simply "delete" one. That is a problem: if you change your password because it was stolen, tokens the thief already holds keep working until they expire.

RuWork solves this with a counter:

```js
// utils/account.js
export function isTokenVersionCurrent(claims, account) {
    return Number(claims?.tv || 0) === Number(account?.tokenVersion || 0);
}
```

Each account stores `tokenVersion`. Every token embeds it as `tv`. Password change, password reset, and sign-out all increment the stored counter. Every previously issued token now carries a stale `tv` and is rejected with `401 TOKEN_REVOKED`.

Why `|| 0` on both sides? Tokens issued before Phase 10 have no `tv` claim, and accounts default to `0`. Treating a missing claim as `0` means the upgrade did not sign out every existing user.

Where it is enforced: inside the three guards that already re-read the account, so revocation costs **zero extra database queries**.

5.5 Four different questions

These are constantly confused. They are four separate checks:

CONCEPT	QUESTION	FAILURE	WHERE
Authentication	Who are you?	`401`	`authenticateToken()`
Authorization	May your *role* do this?	`403`	`isStudent` / `isJobProvider` / `isAdmin`
Eligibility	Does your live account still satisfy the business rules?	`403`	`requireEligibleRuhunaStudent()` etc.
Moderation	Has an Admin suspended you?	`403`	checked inside the eligibility guards and at login

A student can be authenticated (valid token), authorized (role is `student`), and still be blocked because they are suspended.

§ 6

Authentication middleware

`middlewears/authMiddleware.js` is the security heart of the backend.

6.1 `authenticateToken(req, res, next)`

Reads `Authorization`, requires the `Bearer` scheme, verifies the signature, and on success sets `req.user` to the decoded claims.

What it trusts: only the cryptographic signature. **What it does *not* trust:** that the account still exists, is approved, is unsuspended, or that the token has not been revoked. It cannot know — those facts live in the database. **On failure:** `401` and the chain stops. The controller never runs.

6.2 `authorizeRoles(...allowedRoles)`

A **factory** — a function that returns a middleware:

```js
export function authorizeRoles(...allowedRoles) {
    return function authorizeRole(req, res, next) {
        if (!req.user || !allowedRoles.includes(req.user.role)) {
            return res.status(403).json({ error: "You are not authorized to access this resource" });
        }
        return next();
    };
}

export const isAdmin = authorizeRoles(ADMIN_ROLE);
export const isJobProvider = authorizeRoles(JOB_PROVIDER_ROLE);
export const isStudent = authorizeRoles(STUDENT_ROLE);
```

Written once, configured three ways. This is a common and useful JavaScript pattern — see also `changePassword("student")` in the password controller.

This is a **cheap** check: it reads the already-verified claim, no database call. It filters out obviously wrong roles before the expensive guard runs.

6.3 The authoritative guards

Three guards, one shape. `requireEligibleRuhunaStudent()`:

```js
const student = await User.findById(req.user?.sub);
if (!student ||
    student.role !== STUDENT_ROLE ||
    !isAllowedStudentEmail(student.email) ||
    student.university !== UNIVERSITY_NAME ||
    !student.isEmailVerified ||
    student.accountStatus !== "approved" ||
    student.moderationStatus === "suspended") {
    return res.status(403).json({ error: "…", code: "STUDENT_NOT_ELIGIBLE" });
}
if (!isTokenVersionCurrent(req.user, student)) {
    return res.status(401).json(REVOKED_TOKEN_RESPONSE);
}
req.studentAccount = student;
return next();
```

`requireApprovedJobProvider()` mirrors it for providers; `requireAdminAccount()` re-reads the Admin and confirms the role.

The key output: `req.studentAccount` / `req.jobProviderAccount` / `req.adminAccount` — the **live document**. Controllers use these, never `req.user`, whenever they need real account data.

6.4 Why re-read the database at all?

This is the question worth understanding properly. The JWT already says `role: "student"` and carries the id — why spend a query?

Because a JWT is a photograph, not a live feed. It captures the account state at the moment of login and cannot change afterwards. In a 24-hour window a lot can happen:

EVENT AFTER THE TOKEN WAS ISSUED	TOKEN STILL SAYS	REALITY
Admin suspends the student	eligible	must be blocked
Admin rejects the registration	approved	must be blocked
Account deleted	exists	gone
Password changed on another device	valid session	must be revoked

Without the re-read, a suspended student keeps full access until their token expires — up to a day of unrestricted use after being banned.

> **Security reason:** the JWT answers *"who claims to be making this request?"* The database answers *"is that account allowed to do this **right now**?"* You need both. The cost is one indexed `findById` — microseconds — and it is what makes suspension, rejection, and revocation take effect immediately.

6.5 Middleware chains

```JS
adminRouter.use(authenticateToken, isAdmin, requireAdminAccount);
```

```
        Request → authenticateToken → isAdmin → requireAdminAccount → controller
                      |                  |              |
                 401 if bad          403 if not     403 if no Admin record
                 token               admin role      401 if tv is stale
```

Each layer is cheaper than the next: signature check (no I/O) → role check (no I/O) → database read. Rejecting early avoids paying for later layers.

Note `adminRouter.use(...)` is placed **after** `POST /login` and before everything else, so every subsequent admin route inherits all three guards automatically. Adding a new admin endpoint cannot accidentally be left unprotected.

6.6 `requireCommunicationParticipant`

Messages and notifications are for Students *and* Providers, so this guard dispatches on role and delegates to the matching authoritative guard, then normalises the result:

```JS
req.communicationParticipant = { type: "student", id: …, account: … };
```

Downstream controllers work with one shape regardless of role. Any other role (an Admin, for instance) is rejected with `403`.

> **Security reason:** this single guard is why an Admin token cannot read private messages. There is no admin branch — the function ends in a rejection.

§ 7

Email verification

Implemented in `utils/emailVerification.js`, `controllers/emailVerificationController.js`, and `utils/emailService.js`.

7.1 Issuing a token

```JS
export function issueVerificationToken(account, now = new Date()) {
    const rawToken = crypto.randomBytes(32).toString("hex");   // 64 hex chars
    account.emailVerificationTokenHash = hashVerificationToken(rawToken);  // SHA-256
    account.emailVerificationExpiresAt = new Date(now.getTime() + expiryMs);
    account.verificationEmailSentAt = now;
    return rawToken;                    // returned ONCE, for the email only
}
```

- **32 random bytes** from `crypto.randomBytes` — cryptographically secure, not `Math.random()`. 2^{256} possibilities makes guessing hopeless.
- **Only the SHA-256 hash is stored.**
- **An expiry** bounds the window (30 minutes by default).

> **Security reason — why hash a verification token?** The token *is* a credential: whoever holds it can verify that email address. If the database leaked and tokens were stored raw, an attacker could use every unexpired one. Storing only the hash makes a leaked dump useless, exactly as with passwords. SHA-256 (fast) is appropriate here rather than bcrypt (slow) because the token has 256 bits of entropy — there is nothing to brute-force.

7.2 Verifying

```JS
export async function findVerificationAccount(Model, rawToken, now = new Date()) {
    if (!isVerificationTokenFormatValid(rawToken)) return null;   // must be 64 hex chars
    return Model.findOne({
        emailVerificationTokenHash: hashVerificationToken(rawToken),
        emailVerificationExpiresAt: { $gt: now }                  // expiry enforced IN the query
    }).select(PRIVATE_VERIFICATION_FIELDS);
}
```

Two subtleties worth copying elsewhere:

1. **Format-check first.** A malformed token is rejected without touching the database.
2. **Expiry is part of the query, not a later `if`.** An expired token simply does not match. There is no window in which code could forget to check.

On success the controller sets `isEmailVerified = true` and calls `clearVerificationToken()` — making the link **single-use**. Verification deliberately does **not** change `accountStatus`; approval remains a separate human decision.

7.3 Resending and the cooldown

```JS
export function getVerificationResendWaitSeconds(account, now = new Date()) {
    if (!account.verificationEmailSentAt) return 0;
    const availableAt = account.verificationEmailSentAt.getTime() + cooldownMs;
    return Math.max(0, Math.ceil((availableAt - now.getTime()) / 1000));
}
```

Resending issues a fresh token (invalidating the previous one) and enforces a 60-second cooldown stored **in the database**, not in memory.

> **Why persist the cooldown?** *An in-memory timer resets on restart and is not shared between instances. Persisted, it survives both. It protects your SMTP reputation and stops RuWork being used to spam a third party's inbox.*

7.4 Delivery

`utils/emailService.js` lazily builds a Nodemailer transport from `EMAIL_*` variables and throws `"Email service is not configured"` if any are missing. `buildVerificationUrl()` composes `{CLIENT_URL}/verify-email?token=…&type=student|jobProvider`.

Note the direction: the link points at the **frontend**, which reads the query string and calls the API. That is why `CLIENT_URL` must be correct in production — otherwise every verification link points somewhere useless.

§ 8

Password lifecycle

`utils/password.js` + `controllers/passwordController.js`. Four operations, three account types.

8.1 Change password (authenticated)

`PATCH /api/users/password`, `/api/jobProviders/password`, `/api/admin/password`.

1. Reject any body field other than `currentPassword` / `newPassword`.
2. Load the account; verify `currentPassword` with `bcrypt.compare` → `401 CURRENT_PASSWORD_INVALID`.
3. Enforce strength via `getPasswordValidationError` (≥8 chars, ≥1 uppercase, ≥1 digit).
4. **Reject reuse** — `bcrypt.compare(newPassword, account.password)` → `400`.
5. Hash and store.
6. `revokeIssuedTokens(account)` → `tokenVersion++`, `passwordChangedAt = now`.
7. Return a **freshly signed token**.

> **Why return a new token?** *Step 6 invalidates every existing token — including the one the user is holding right now. Without a replacement they would be signed out by their own successful action, which feels like a bug. Returning one new token means "every other session is dead; this device continues."*

Why re-verify the current password if they are already authenticated? Because an unattended logged-in laptop should not be enough to take over an account permanently. It also proves the *person* is present, not just the session.

8.2 Forgot password (unauthenticated, Student/Provider only)

`POST /api/users/password/forgot`, `/api/jobProviders/password/forgot`.

Every outcome returns the same body:

```js
const GENERIC_RESET_RESPONSE = {
    message: "If an account exists for that address, a password reset link has been sent."
};
```

Unknown address, suspended account, active cooldown, successful send — all identical, all `200`.

> **Security reason — enumeration attacks.** If an unknown address returned "no such account" and a real one returned "email sent", the endpoint becomes a *membership oracle*. Feed it a leaked email list and you learn exactly who has a RuWork account. That is a privacy breach, and it hands attackers a validated target list for phishing and credential stuffing. Identical responses close it.

A failed email send rolls the token back **and still returns the generic body** — even delivery timing must not leak account existence.

8.3 Reset password

```js
const account = await findAccountByResetToken(definition.Model, body.token);
if (!account) return res.status(400).json({ error: "…invalid or has expired.", code: "RESET_TOKEN_INVALID"
});
account.password = await bcrypt.hash(body.newPassword, 10);
clearResetToken(account);
account.passwordResetRequestedAt = undefined;
revokeIssuedTokens(account);
await account.save();
return res.json({ message: "Password reset successfully. Please sign in with your new password." });
```

Same hashed-token pattern as verification: format check, hash lookup, expiry in the query, consumed on use.

> **Why return no token here?** *A reset happens when someone may have lost control of their account. Forcing a fresh login re-runs every login gate — verification, approval, and moderation — so a suspended or rejected account cannot be revived through the reset flow.*

8.4 Sign out

Increments `tokenVersion`, ending **all** sessions for that account.

8.5 Why Admin differs

Admins get change and logout, but **no** `password/forgot` or `password/reset`. This is asserted by a test in `tests/phase10.test.js`.

> **Security reason:** self-service reset depends entirely on email. If an Admin's mailbox were compromised, an attacker could seize full platform control without ever knowing a password. Admins are provisioned by an operator with server access, so recovery is also an operator action. Removing the automated path removes the attack.

§ 9

Student registration and account lifecycle

Step 1 — `POST /api/users` → `registerUser()`

```
sensitiveRateLimiter          (max 20/hour — stops mass account creation)
   ↓
getPlatformSettings()         → 403 REGISTRATION_CLOSED if studentRegistrationOpen is false
   ↓
normalizeEmail(body.email)    → trimmed, lowercased
isAllowedStudentEmail(email)  → 400 unless the domain is exactly ruh.ac.lk
   ↓
reject a conflicting university value → 400
   ↓
getPasswordValidationError()  → 400 if weak
   ↓
bcrypt.hash(password, 10)
   ↓
new User({ …explicit fields only…, university: UNIVERSITY_NAME,
           isEmailVerified: false, accountStatus: "pending", role: STUDENT_ROLE })
   ↓
issueVerificationToken(newUser)  → save()
   ↓
emailDelivery.sendVerificationEmail(...)
   ↓
201  { message, accountStatus: "pending", isEmailVerified: false }   ← no JWT
```

Three things to notice.

The document is built field by field. The controller never does `new User(req.body)`. A **mass-assignment** attack — posting `{"role":"admin","accountStatus":"approved"}` — has nothing to latch onto, because those fields are set from constants. (The `immutable` schema flags are the second layer.)

No JWT is returned. Registration is not authentication. The account cannot log in yet.

Email failure is handled honestly:

```JS
catch (error) {
    await allowImmediateVerificationRetry(newUser).catch(() => {});
    return res.status(503).json({ error: "Account created, but the verification email could not be sent…",
                                  code: "VERIFICATION_EMAIL_NOT_SENT" });
}
```

The account survives, the cooldown is cleared so the user can retry immediately, and the response says exactly what happened.

Duplicate email → Mongo `11000` → `409 "An account already uses this email"`.

Step 2 — verification

`GET /api/users/verify-email/:token` → `isEmailVerified = true`. `accountStatus` unchanged.

Step 3 — Admin approval

`PATCH /api/admin/registrations/student/:id/approve` → `accountStatus = "approved"`, records `reviewedAt` / `reviewedBy`, writes an audit record.

Approval **requires** `isEmailVerified === true` → otherwise `409 EMAIL_NOT_VERIFIED`. An already-reviewed registration → `409 REGISTRATION_ALREADY_REVIEWED`.

Step 4 — login

`POST /api/users/login` → `loginUser()`, checked in this order:

```
email + password present?           → 400
User.findOne({ email })
bcrypt.compare                      → 401 "Invalid email or password"
role/university/domain still valid? → 403 STUDENT_NOT_ELIGIBLE
isEmailVerified?                    → 403 EMAIL_NOT_VERIFIED
accountStatus === "approved"?       → 403 ACCOUNT_PENDING | ACCOUNT_REJECTED
moderationStatus !== "suspended"?   → 403 ACCOUNT_SUSPENDED
                                    → 200 { token }
```

> **Why one message for a wrong email *and* a wrong password?** *`"Invalid email or password"` prevents enumeration — the same reasoning as §8.2. Note the later messages are deliberately specific, because by then the password has already proven the caller owns the account.*

Two gates, not one. Email verification proves control of the inbox; Admin approval is a human decision. Passing one does not imply the other.

§ 10

Job Provider registration and lifecycle

Structurally identical to the Student flow, with four differences.

1. **No domain restriction** — `hasBasicEmailFormat()` instead of `isAllowedStudentEmail()`.
2. **`companyEmail` is the canonical field** everywhere.
3. **Company data** is captured: name, address, size, industry, optional website, description.
4. **`providerRegistrationOpen`** is the settings gate.

`registerJobProvider()` and `loginJobProvider()` live in `controllers/jobProviderController.js`; the guard is `requireApprovedJobProvider()`.

Once approved, a provider can post jobs, manage applicants, message students, and read reviews about them.

Provider suspension — what happens to their jobs

`PATCH /api/admin/providers/:id/moderation` with `status: "suspended"`:

```js
account.moderationStatus = "suspended";
account.moderationReason = reason;          // 5-500 characters, required
account.moderatedAt = moderatedAt;
account.moderatedBy = req.user.sub;
await account.save();

await Job.updateMany(
    { jobProviderId: account._id },
    { $set: { providerSuspendedAt: moderatedAt } }
);
```

Consequences:

AREA	EFFECT
Provider login	Blocked (`ACCOUNT_SUSPENDED`)
Provider API access	Blocked by `requireApprovedJobProvider`
Their jobs in public browse	Hidden (`providerSuspendedAt: null` is required)
Existing applications	**Preserved**
Existing messages/notifications	**Preserved**
Reviews about them	**Still visible**

> **Why do reviews stay visible?** *They are Student-authored history about engagements that really happened. Hiding them would let a badly-behaved provider erase legitimate feedback by getting suspended — the opposite of the intended incentive.*

Restoring sets `moderationStatus = "active"` and `providerSuspendedAt = null`. Fully reversible.

§ 11

Admin system

11.1 Private provisioning

There is no `POST /api/admin` route. The only creation path is `scripts/createAdmin.js`, run as `npm run create-admin`, which reads `ADMIN_FIRST_NAME`, `ADMIN_LAST_NAME`, `ADMIN_EMAIL`, `ADMIN_PASSWORD` from the environment, validates them, refuses duplicates, hashes the password, and **never prints the plaintext**.

> **Security reason:** a public admin-registration endpoint is a total compromise waiting to be found. Requiring server access to create an admin means an attacker must already own the server — at which point the endpoint is irrelevant. A test in `tests/phase2.test.js` asserts `POST /` does not exist on the admin router.

11.2 Blanket protection

```js
adminRouter.post("/login", authRateLimiter, loginAdmin);   // public
adminRouter.use(authenticateToken, isAdmin, requireAdminAccount);   // everything below is guarded
```

All 20 subsequent admin endpoints inherit three layers automatically.

11.3 Capabilities

AREA	ENDPOINTS	NOTES
Dashboard	`GET /api/admin/dashboard`	Counts only — **no message content**
Registration Reviews	`GET /registrations`, `GET /registrations/:type/:id`, `PATCH …/approve`, `PATCH …/reject`	Reuses the Phase 2 logic
Students	`GET /students`, `GET /students/:id`, `PATCH /students/:id/moderation`	Search, filter, paginate, suspend/restore
Providers	`GET /providers`, `GET /providers/:id`, `PATCH /providers/:id/moderation`	Suspension cascades to jobs
Jobs	`GET /jobs`, `GET /jobs/:id`, `PATCH /jobs/:id/moderation`	`visible ⇄ hidden`
Reviews	`GET /reviews`, `GET /reviews/:id`, `PATCH /reviews/:id/moderation`, `DELETE /reviews/:id`	Hide/restore, plus permanent delete
Settings	`GET /settings`, `PATCH /settings`	Three booleans
Audits	`GET /audits`	Read-only
Own account	`PATCH /password`, `POST /logout`	No reset path

11.4 Shared admin safety helpers (`utils/admin.js`)

HELPER	PURPOSE
`adminPagination(query, defaultLimit, { maxPage })`	Scalar-only, page 1–10000, limit 1–50
`boundedSearch(value, max)`	Must be a string, ≤80 chars, whitespace collapsed
`escapeAdminRegex(value)`	Escapes regex metacharacters
`moderationReason(value, { required })`	5–500 chars when required
`assertOnlyFields(body, allowed)`	Rejects any unexpected body field
`createAdminAudit({...})`	Validates and writes an audit record
`getPlatformSettings()`	Reads settings with defaults

Two of these deserve a closer look.

`escapeAdminRegex` — searches build a `$regex`. Without escaping, a search for `.*` matches everything and a crafted pattern can cause catastrophic backtracking (a **ReDoS** — the server pins a CPU core for minutes on one request). Escaping makes the input a literal string.

`adminPagination`'s scalar guard — `Number(["2"])` evaluates to `2` in JavaScript, so `?page[]=2` would sneak through a naive numeric check. The helper rejects non-string/non-number input *before* coercion:

```JS
function paginationNumber(value, fallback) {
    if (value === undefined) return fallback;
    if (typeof value !== "string" && typeof value !== "number") return NaN;
    return Number(value);
}
```

11.5 Why moderation is reversible, never destructive

Every moderation action is a status flip.

- **Mistakes happen.** A wrongly suspended account is restored with one click. A deleted one is gone forever.
- **History must survive.** Deleting a job would orphan its applications, messages, and reviews — breaking every student's job history.
- **Disputes need evidence.** "Why was I suspended?" is answerable because the record and its `moderationReason` still exist.
- **Audits must stay meaningful.** An audit pointing at a deleted `entityId` is useless.

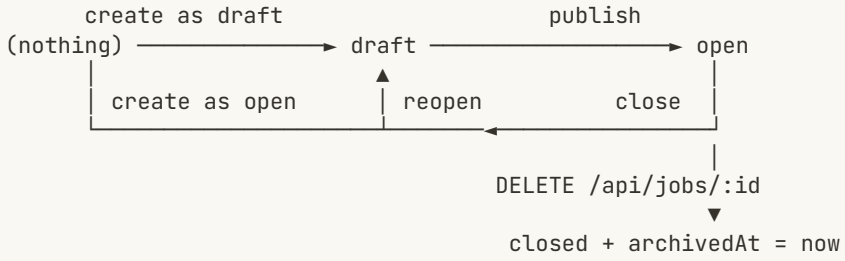
The one deliberate exception is `DELETE /api/admin/reviews/:id`, retained from Phase 7 for content that must not persist at all (e.g. abuse or personal data). It writes a `REVIEW_DELETED` audit and recalculates aggregates.

§ 12

Job system

`controllers/jobController.js` + `models/job.js` + `utils/job.js`.

12.1 Lifecycle



- **draft** — private to the provider; never in public browse.
- **open** — publicly visible and accepting applications.
- **closed** — visible to the provider, not accepting applications.
- **archived** — `archivedAt` set. Gone from browse *and* from My Jobs, but all history is intact.

> **Why does `DELETE` archive instead of erase?** *(The plan calls this "Option B".) Erasing a job would orphan its applications, messages, and reviews, corrupting every related student's history. Archiving satisfies "remove it from my list" while keeping referential integrity.* **Nothing in RuWork hard-deletes a job.**

12.2 The public query — five conditions

```js
export function buildPublicJobQuery(query = {}, now = new Date()) {
    const filter = {
        archivedAt: null,                       // 1 not archived
        status: "open",                         // 2 published
        moderationStatus: { $ne: "hidden" },    // 3 not hidden by an Admin
        providerSuspendedAt: null,              // 4 provider not suspended
        applicationDeadline: { $gt: now }       // 5 deadline in the future
    };
    …
}
```

All five must hold. The identical conditions are repeated in `getJob()` (details) and in `applicationController.applyToJob()` — **defence in depth**: even if a job id is guessed or bookmarked, the detail endpoint hides it, and even then an application cannot be created.

Filters supported: bounded text search (`$text`), category, escaped location and skill, suitable year, budget type, price range, plus whitelisted sorts (`newest`, `oldest`, `price-low`, `price-high`, `rating`).

> **Why whitelist sorts?** *Accepting a raw sort field lets a caller sort by `priceAmount` (deliberately `select: false`) or force an unindexed sort that scans the whole collection.*

12.3 System-field protection

```js
const SYSTEM_FIELDS = [
    "jobProviderId", "companyName", "currency", "priceAmount",
    "averageRating", "reviewCount", "archivedAt", "createdAt", "updatedAt",
    "moderationStatus", "moderationReason", "moderatedAt", "moderatedBy", "providerSuspendedAt"
];
function assertNoSystemFields(body = {}) {
    const supplied = SYSTEM_FIELDS.find((field) => Object.hasOwn(body, field));
    if (supplied) throw new JobInputError(`${supplied} cannot be set directly`);
}
```

Run on **both** create and update. Ownership comes from `req.jobProviderAccount._id`, never from the body.

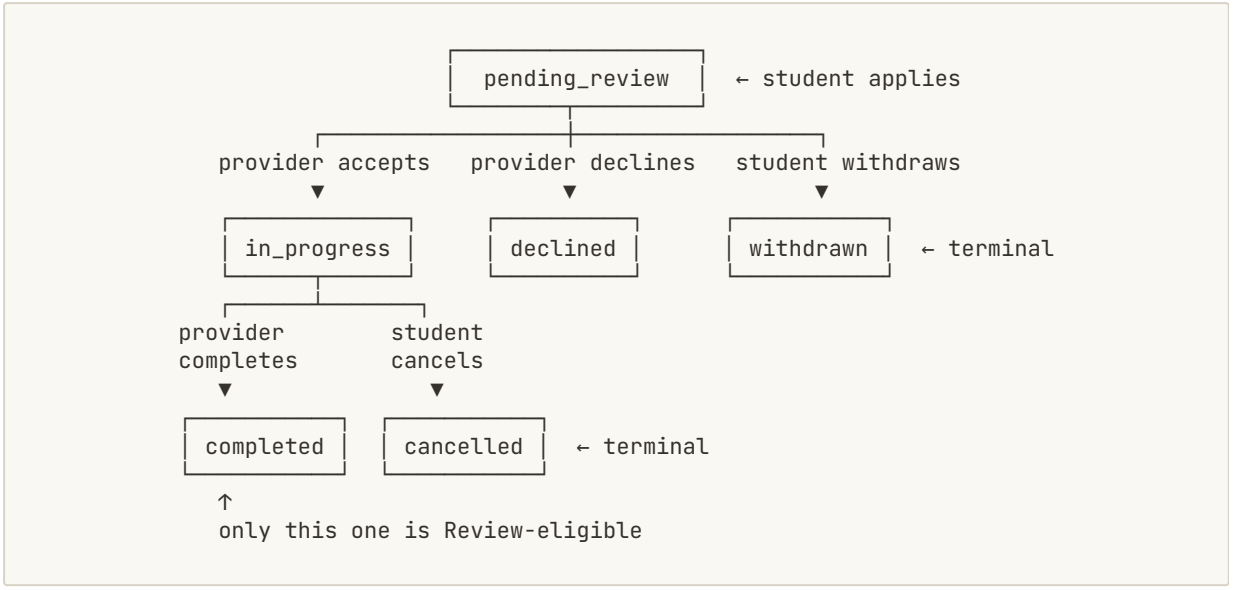
> **Security reason:** without this a provider could `PATCH { "moderationStatus": "visible" }` and un-hide a job an Admin had hidden, or `PATCH { "averageRating": 5 }` and fake a perfect score. The update path additionally assigns only `EDITABLE_FIELDS` plus a validated status transition, so publishing or reopening a job **cannot** clear an Admin's hide.

Ownership is enforced by `ownsJob()` comparing `job.jobProviderId` with the authenticated provider's id → `403`.

§ 13

Application system

13.1 The state machine



Encoded declaratively in `utils/application.js`:

```js
const TRANSITIONS = {
    student:  { pending_review: ["withdrawn"],            in_progress: ["cancelled"] },
    provider: { pending_review: ["in_progress","declined"], in_progress: ["completed"] }
};

export function assertApplicationTransition(currentStatus, nextStatus, actor) {
    if (!APPLICATION_STATUSES.includes(nextStatus) || !TRANSITIONS[actor]?.
[currentStatus]?.includes(nextStatus)) {
        throw new ApplicationConflictError(`Application cannot transition from ${currentStatus} to
${nextStatus}`);
    }
}
```

> **Why a table rather than scattered `if`s?** *The rules are visible in one place, both who and from-what-to-what are checked together, and every illegal move — a provider completing a declined application, a student withdrawing after acceptance — is impossible by construction rather than by remembering to write a check.*

13.2 Creating an application

`POST /api/jobs/:jobId/applications` → `applyToJob()`.

> ⚠ Note the URL: creation lives on the **Job** router (`routes/jobRouter.js`), not `/api/applications`. See *Overview §11.1*.

```
authenticateToken → isStudent → requireEligibleRuhunaStudent
   ↓ (six eligibility conditions re-checked against the live account)
validate the note (20–1000 chars)
   ↓
load the Job with the SAME five public conditions
   ↓  (archived / draft / closed / expired / hidden / suspended-provider → 404)
snapshot the pricing from the Job
   ↓
Application.create({ jobId, studentId, jobProviderId, applicationNote, budgetType, original* })
   ↓  duplicate → Mongo 11000 → 409 APPLICATION_ALREADY_EXISTS
notifyApplication(NEW_APPLICATION → provider)
   ↓
201
```

Identities come from `req.studentAccount._id` and the Job's `jobProviderId` — never from the body.

13.3 Provider decisions

`PATCH /api/applications/provider/:id/accept` requires the correct approved price for the budget type (`approvedHourlyRate` or `approvedBudget`). Ownership is verified **through the Job**, not a client-supplied provider id.

13.4 Pricing is information only

`originalHourlyRate` / `originalBudget` are immutable snapshots; `approvedHourlyRate` / `approvedBudget` record what was agreed.

> **There is no payment processing.** No gateway, no transaction record, no Paid/Pending status. These fields exist so both parties can see what was agreed. Money changes hands directly between them, outside RuWork.

§ 14

Review and rating system

14.1 Rules

- **Who:** the Student who owned the engagement, and only them.
- **What:** an application with status exactly `completed`.
- **How many:** one active review per application (unique index).
- **Rating:** an integer 1–5.
- **Comment:** optional, ≤1000 characters, plain text.

Archived jobs remain reviewable through their completed application. Deleting your review lets you write a new one.

14.2 `createReview()` — trust the Application, not the client

The client sends only `applicationId`, `rating`, and `comment`. Everything else is derived:

```js
review = new Review({
    applicationId: application._id,
    jobId:         application.jobId,          // from the Application
    studentId:     application.studentId,      // from the Application
    jobProviderId: application.jobProviderId,  // from the Application
    rating:  reviewRating(req.body.rating),
    comment: reviewComment(req.body.comment)
});
```

> **Security reason:** if the client supplied `jobProviderId`, anyone could post a 1-star review against a competitor they had never worked with. Deriving all identities from the authoritative Application makes that structurally impossible.

Ownership is checked (`application.studentId` vs `req.studentAccount._id` → `403`), status must be `completed` (→ `409`), and Job↔Provider consistency is re-verified.

14.3 `utils/ratingAggregates.js`

Each Job and each Provider stores `averageRating` and `reviewCount` — **denormalised** copies.

> **Why denormalise?** *A job list showing 20 cards would otherwise need 20 aggregation queries just to display stars. Storing the summary makes the list a single indexed read. The cost is that the copy must be kept correct.*

```JS
async function ratingSummary(match) {
    const [summary] = await Review.aggregate([
        { $match: { ...match, moderationStatus: { $ne: "hidden" } } },
        { $group: { _id: null, averageRating: { $avg: "$rating" }, reviewCount: { $sum: 1 } } }
    ]);
    return summary
        ? { averageRating: roundedRating(summary.averageRating), reviewCount: summary.reviewCount }
        : { averageRating: null, reviewCount: 0 };
}
```

`recalculateReviewAggregates(jobId, jobProviderId)` recomputes both in parallel and is called after **every** path that changes reviews: create, student delete, admin delete, and moderation hide/restore.

Three properties worth naming:

1. **`moderationStatus: { $ne: "hidden" }` is inside the aggregation.** This is why hiding a review immediately fixes the rating — the hidden document is excluded from the recount. Without it, a hidden abusive 1-star review would keep dragging the provider's score down forever.
2. **It is idempotent.** It recomputes from scratch rather than incrementing, so running it twice is harmless and a missed run is repaired by the next one.
3. **Empty means `null`, not `0`.** A provider with no reviews shows "No ratings yet" rather than a misleading zero.

Compensating rollback: local/test MongoDB may not be a replica set, so transactions are unavailable. Instead, if aggregate recalculation fails after a review was saved, the review is rolled back and a safe `500` is returned — the database is never left inconsistent.

§ 15

Messaging

15.1 Why there is no Conversation model

A conversation is *derived*: all messages sharing an `applicationId`, ordered by `createdAt`.

> **Why not store one?** *A Conversation document would duplicate facts already in the Application (who the two participants are, which job) and would need to stay in sync — a classic source of bugs. The Application already* **is** *the relationship. Adding a Conversation would add work and risk without adding information.*

Conversation *summaries* are built with an aggregation over messages, then hydrated with a small number of batch queries — **not** one query per conversation (the "N+1 query" problem, where showing 50 conversations costs 51 round trips).

15.2 Authorization

`messageRouter.use(authenticateToken, requireCommunicationParticipant)` — so every message route requires an eligible Student or approved Provider. Admins are rejected outright.

`authorizedContext(applicationId, participant)` then loads the Application and confirms the caller is genuinely one of its two parties → `404` if it does not exist, `403` if they are not a participant.

15.3 Sending — identities are derived, never supplied

```js
const senderType = participant.type;
const receiverType = senderType === "student" ? "jobProvider" : "student";
const receiverId   = senderType === "student" ? application.jobProviderId : application.studentId;
```

The client supplies only `applicationId`, `content`, and optionally `includeContactNumber`. **The recipient cannot be chosen.** It is computed from the Application, so RuWork can never be used to message an arbitrary account.

15.4 Contact sharing

```js
sharedContactNumber: senderType === "student" && req.body?.includeContactNumber
    ? participant.account.phoneNumber      // re-read from the authenticated profile
    : null
```

The number is taken from the **server's copy** of the student's profile, not from the request. `includeContactNumber` is a boolean flag only.

> **Security reason:** if the client could send a number, a student could attach someone else's phone number to a message — harassment by proxy. And because only students may set it, a provider cannot harvest or fabricate contact details.

`sharedContactNumber` is `immutable`: once sent, it cannot be altered.

15.5 Read state

Opening a thread marks only **received** unread messages as read — you can never mark your own message read on the recipient's behalf.

15.6 Why Admins cannot read messages

Three independent reasons, any one of which suffices:

1. `requireCommunicationParticipant` has no admin branch — it ends in `403`.
2. No admin router endpoint touches the `Message` model except `countDocuments`.
3. The Admin dashboard exposes only `communication.messages` — a number.

> **Design reason:** private conversations between two adults about work are not the platform's business. Reporting counts supports operational insight ("is messaging being used?") without surveillance.

§ 16

Notifications

16.1 The seven triggers

TYPE	FIRED WHEN	RECIPIENT
`NEW_APPLICATION`	Student applies	Provider
`APPLICATION_ACCEPTED`	Provider accepts	Student
`APPLICATION_DECLINED`	Provider declines	Student
`APPLICATION_WITHDRAWN`	Student withdraws	Provider
`APPLICATION_CANCELLED`	Student cancels	Provider
`APPLICATION_COMPLETED`	Provider completes	Student
`NEW_MESSAGE`	A message is sent	The receiver

Application events go through `notifyApplication()` in `controllers/applicationController.js`; messages call `createNotificationSafely()` directly.

16.2 Best-effort by design

```js
export async function createNotificationSafely(details) {
    if (isTestEnvironment() && mongoose.connection.readyState === 0 && Notification.create === originalNotificationCreate) {
        return null;
    }
    try {
        return await createNotification(details);
    } catch (error) {
        logger.warn("Notification creation failed after a successful business action", { name: error?.name });
        return null;
    }
}
```

Notifications are created **after** the core action has already succeeded, and a failure is swallowed and logged.

> **Why?** *The notification is a convenience; the application is the real outcome. If a notification write failed and that failure propagated, the student would see "application failed" when their application had in fact been created — and retrying would hit the duplicate index and confuse them further. Better a missing notification than a lie about the core action.*

Note this is a genuine trade-off: notifications can be silently lost. That is accepted because they are not the system of record — the Application is.

16.3 Ownership

Every notification operation is recipient-scoped. `markNotificationRead` matches on both the id **and** the recipient, so you cannot mark someone else's notification read even by guessing its id.

§ 17

Platform Settings

17.1 The singleton

```js
singletonKey: { type: String, enum: ["platform"], default: "platform", unique: true, immutable: true }
```

An enum of exactly one value, plus a unique index, means **at most one settings document can ever exist.** No "which row is the real config?" ambiguity.

17.2 The three settings

SETTING	EFFECT WHEN FALSE	ENFORCED IN
`studentRegistrationOpen`	`POST /api/users` → `403 REGISTRATION_CLOSED`	`userController.registerUser()`
`providerRegistrationOpen`	`POST /api/jobProviders` → `403 REGISTRATION_CLOSED`	`jobProviderController.registerJobProvider()`
`jobPostingOpen`	`POST /api/jobs` → `403 JOB_POSTING_CLOSED`	`jobController.createJob()`

Server-authoritative. The frontend does not merely hide a button — the API refuses the action. Hiding a button stops honest users; refusing the request stops everyone.

17.3 Why an allowlist and not a key/value store

```js
export const SETTINGS_DEFAULTS = Object.freeze({
    studentRegistrationOpen: true, providerRegistrationOpen: true, jobPostingOpen: true
});
export const SETTING_FIELDS = Object.keys(SETTINGS_DEFAULTS);
```

`updateAdminSettings()` calls `assertOnlyFields(req.body, SETTING_FIELDS)` and requires every value to be a literal boolean.

> **Security reason — why generic settings tables are dangerous.** A `{ key, value }` store invites storing operational secrets: `jwtSecret`, `mongoUri`, `smtpPassword`. Once a secret lives in the database it appears in backups, in admin UIs, and in any endpoint that lists settings — and a single compromised Admin account leaks the whole system. Restricting settings to three typed business booleans means **there is nothing sensitive to leak**. The Settings page states this explicitly to the Admin.

Every change writes a `SETTINGS_UPDATED` audit with a `{ from, to }` diff.

§ 18

Admin Audit Trail

18.1 What is recorded

```JS
{
  adminId:    ObjectId,   // WHO  — from the verified JWT, never the body
  action:     String,     // WHAT — one of 12
  entityType: String,     // on what kind of thing — one of 6
  entityId:   ObjectId,   // which one
  metadata:   Mixed,      // context, ≤1500 serialised characters
  createdAt:  Date        // WHEN — server clock
}
```

Actions: `REGISTRATION_APPROVED`, `REGISTRATION_REJECTED`, `STUDENT_SUSPENDED`, `STUDENT_RESTORED`, `PROVIDER_SUSPENDED`, `PROVIDER_RESTORED`, `JOB_HIDDEN`, `JOB_RESTORED`, `REVIEW_HIDDEN`, `REVIEW_RESTORED`, `REVIEW_DELETED`, `SETTINGS_UPDATED`. Entity types: `registration`, `student`, `jobProvider`, `job`, `review`, `settings`.

Examples:

```JS
// Suspending a student
{ action: "STUDENT_SUSPENDED", entityType: "student", entityId: <studentId>,
  metadata: { from: "active", to: "suspended", reason: "Repeated policy breach" } }

// Changing a setting
{ action: "SETTINGS_UPDATED", entityType: "settings", entityId: <settingsId>,
  metadata: { changes: { jobPostingOpen: { from: true, to: false } } } }
```

18.2 Why the identity must come from the server

```JS
await createAdminAudit({
    adminId: req.user.sub,     // ← the verified JWT subject
    …
});
```

> **Security reason:** if `adminId` came from the request body, a malicious Admin could log their actions under a colleague's identity. Framing someone is exactly the kind of abuse an audit trail exists to prevent. The same applies to the timestamp: `createdAt` is the server clock, so a client cannot backdate an entry.

`createAdminAudit()` re-validates both ObjectIds and both enumerations before writing, so a bug elsewhere cannot produce a meaningless record.

18.3 Immutability

Every field is `immutable` and the schema has `updatedAt: false`. There is **no** create, update, or delete API — the only exposure is `GET /api/admin/audits`, which is paginated and filterable.

18.4 Compensating rollback

```JS
account.moderationStatus = req.body.status;
await account.save();
try {
    await createAdminAudit({ … });
} catch (error) {
    Object.assign(account, previous);      // put it back
    await account.save().catch(() => {});
    throw error;
}
```

> **Why undo the action if only the audit failed?** *An unaudited administrative action is worse than no action. If audits could silently fail, "there is no record of that suspension" becomes ambiguous — did it not happen, or did the log fail? Reversing the change preserves the invariant* **every completed admin action has exactly one audit record**. *Provider suspension also reverses its `Job.updateMany` cascade.*

§ 19

Security middleware

`middlewears/security.js`.

19.1 Helmet — security headers

```js
helmet({
    contentSecurityPolicy: { useDefaults: false, directives: {
        "default-src": ["'none'"], "frame-ancestors": ["'none'"],
        "base-uri": ["'none'"], "form-action": ["'none'"] } },
    crossOriginResourcePolicy: { policy: "same-site" },
    referrerPolicy: { policy: "no-referrer" },
    hsts: isProduction() ? { maxAge: 31536000, includeSubDomains: true, preload: false } : false
})
```

HEADER	PROTECTS AGAINST
`Content-Security-Policy: default-src 'none'`	If a response were ever misinterpreted as HTML, the browser will load **no** scripts, styles, or images from it
`frame-ancestors 'none'`	**Clickjacking** — nobody can embed the API in an `<iframe>` and trick users into clicking invisible controls
`X-Content-Type-Options: nosniff`	**MIME sniffing** — stops a browser guessing that a JSON response is really HTML/JS
`Referrer-Policy: no-referrer`	Stops URLs (which may contain a reset token) leaking in the `Referer` header
`Strict-Transport-Security` (production only)	Forces HTTPS, blocking protocol-downgrade interception

The policy is maximally strict because this API only ever returns JSON — it needs no scripts, styles, or fonts. HSTS is production-only because it would force HTTPS on `localhost` and break development.

19.2 CORS

Same-origin policy is the browser rule that a page from origin A cannot read a response from origin B. Without CORS, a React app on `localhost:5173` could not call an API on `localhost:5000`. **CORS** is how the server says "this specific origin is allowed."

```js
const allowed = getCorsOrigins();
cors({
    origin(origin, callback) {
        if (!origin || allowed.includes(origin.replace(/\/$/, ""))) return callback(null, true);
        return callback(null, false);
    },
    methods: ["GET","POST","PATCH","DELETE","OPTIONS"],
    allowedHeaders: ["Content-Type","Authorization"],
    credentials: false, maxAge: 600
});
```

Origins come from `CORS_ORIGINS` (comma-separated) or fall back to `CLIENT_URL`. Startup validation **rejects a wildcard in production** and requires an explicit `http://`/`https://` scheme.

> **Security reason — why `*` is unsafe in production:** with a wildcard, any website your logged-in user visits could script requests to your API from their browser. The allowlist means only your own frontend can do so.

Why are requests with no `Origin` header allowed? Non-browser callers (curl, health probes, server-to-server) send no `Origin`. Blocking them would break monitoring while providing no security benefit — CORS is a *browser* protection, not an authorization boundary. **Authorization is still the JWT and the role guards**, which apply to every request regardless of origin.

19.3 Rate limiting

Three tiers:

LIMITER	WINDOW	MAX	APPLIED TO
`apiRateLimiter`	15 min	600	Every request globally
`authRateLimiter`	15 min	10	Login, verify-email, password reset/change
`sensitiveRateLimiter`	60 min	20	Registration, resend verification, forgot password

```js
export const authRateLimiter = limiter({
    windowMs: 15 * 60 * 1000, max: 10,
    skipSuccessfulRequests: true,     // ← only failures count
    message: "Too many authentication attempts. Please wait before trying again.",
    code: "AUTH_RATE_LIMITED"
});
```

> **Why `skipSuccessfulRequests` on login?** *Otherwise a shared office IP could lock out honest users who log in normally all day. Counting only failures targets password guessing precisely.*

Why is registration limited more harshly by time? Each registration sends an email. Unlimited registration means unlimited outbound mail — your SMTP provider will suspend the account and RuWork becomes a spam relay.

Health is exempt via `skip`, so an orchestrator can always probe liveness. Limits are disabled under the test gate so the suite is deterministic.

> **Known limitation:** the default store is **in-memory**, so limits are per-process. Two instances behind a load balancer each allow 10 attempts, i.e. 20 total. A multi-instance deployment needs a shared (e.g. Redis) store. Recorded in `PROJECT_PLAN.md` §"Known Phase 10 limitations".

19.4 Body size limit

`express.json({ limit: getJsonBodyLimit() })` — 100 kB by default.

> **Why:** *without a limit, one request with a 2 GB body exhausts server memory — a trivial denial-of-service. RuWork's largest legitimate body is a job posting (a few kB), so 100 kB is generous. Exceeding it produces `413`.*

§ 20

Environment configuration

Every variable in `.env.example`. **`.env` itself is git-ignored and must never be committed.**

VARIABLE	PURPOSE	REQUIRED	SECRET?	NOTES
`NODE_ENV`	Runtime mode	Recommended	No	`production` enables HSTS, JSON logs, strict validation, and forces the test gate off
`PORT`	Listening port	No	No	Defaults to `5000`
`MONGODB_URI`	Database connection string	**Yes, always**	**YES**	Contains credentials. Startup fails without it
`JWT_SECRET`	Key that signs and verifies JWTs	**Yes, always**	**YES**	Must be ≥32 characters in production. Anyone with it can mint valid tokens for any role
`JWT_EXPIRES_IN`	Token lifetime	No	No	Defaults to `1d`
`CLIENT_URL`	Frontend base URL	Production	No	Used to build verification/reset links **and** as the default CORS origin
`CORS_ORIGINS`	Comma-separated allowlist	Optional	No	Falls back to `CLIENT_URL`. Wildcards rejected in production
`TRUST_PROXY`	Trust `X-Forwarded-For`	No	No	`true`/`false`/hop count. Set only when genuinely behind a proxy
`JSON_BODY_LIMIT`	Max body size	No	No	Default `100kb`
`RATE_LIMIT_DISABLED`	Emergency switch	No	No	Leave unset
`ALLOWED_STUDENT_EMAIL_DOMAIN`	Student domain rule	No	No	Default `ruh.ac.lk`
`EMAIL_HOST`	SMTP host	Production	No	
`EMAIL_PORT`	SMTP port	No	No	Default `587`
`EMAIL_SECURE`	TLS on connect	No	No	`true` for port 465
`EMAIL_USER`	SMTP username	Production	**YES**	
`EMAIL_PASSWORD`	SMTP password	Production	**YES**	
`EMAIL_FROM`	From address	Production	No	
`EMAIL_VERIFICATION_EXPIRES_MINUTES`	Verification token lifetime	No	No	Default `30`
`EMAIL_VERIFICATION_RESEND_COOLDOWN_SECONDS`	Resend cooldown	No	No	Default `60`
`PASSWORD_RESET_EXPIRES_MINUTES`	Reset token lifetime	No	No	Default `30`
`PASSWORD_RESET_COOLDOWN_SECONDS`	Reset request cooldown	No	No	Default `60`

VARIABLE	PURPOSE	REQUIRED	SECRET?	NOTES
`RUWORK_TEST_MODE`	Enables test-only fallbacks	Never in production	No	Startup **rejects** it when `NODE_ENV=production`
`ADMIN_FIRST_NAME` / `ADMIN_LAST_NAME` / `ADMIN_EMAIL`	Admin provisioning	Only for `create-admin`	No	
`ADMIN_PASSWORD`	Admin provisioning password	Only for `create-admin`	**YES**	Never printed
`DEMO_PASSWORD`	Seed-script password	Only for `seed:demo`	**YES**	Never printed

`utils/env.js`

One validated place to read configuration, rather than scattering `process.env` reads across the codebase.

```js
export function getEnvironmentProblems() {
    const problems = [];
    if (!read("MONGODB_URI")) problems.push("MONGODB_URI is not configured");
    if (!read("JWT_SECRET"))  problems.push("JWT_SECRET is not configured");
    if (isProduction()) {
        if (read("JWT_SECRET").length < 32) problems.push("JWT_SECRET must be at least 32 characters in
production");
        if (getCorsOrigins().some((o) => o === "*")) problems.push("CORS_ORIGINS must not contain a
wildcard in production");
        …
    }
    return problems;
}
```

> **Security reason:** problems are reported **by name only** — never the value. A crash log that printed the offending `JWT_SECRET` would leak it into log aggregation, screenshots, and support tickets. A test in `tests/phase10.test.js` asserts the value never appears in the output.

`isTestEnvironment()` is the explicit gate that replaced a fragile Phase 9 pattern. It requires `NODE_ENV=test` or `RUWORK_TEST_MODE=true` and returns `false` outright when `NODE_ENV=production`, so a test-only shortcut can never activate in a live deployment.

§ 21

Centralized error handling

`middlewears/errorHandler.js`.

21.1 `requireObjectBody`

```js
if (["POST","PATCH","PUT"].includes(req.method) && req.body == undefined) {
    const invalid = req.body == null || typeof req.body != "object" || Array.isArray(req.body);
    if (invalid) return res.status(400).json({ error: "Request body must be a JSON object" });
}
```

> **Why:** *`[1,2,3]` and `"hello"` are valid JSON. Controllers assume an object and do `Object.hasOwn(body, field)`, which behaves unpredictably on arrays and strings. Rejecting once, centrally, means no controller needs to defend itself.*

21.2 The error map

CONDITION	STATUS	RESPONSE
`entity.parse.failed` / `SyntaxError`	400	`INVALID_JSON`
`entity.too.large`	413	`PAYLOAD_TOO_LARGE`
Mongoose `ValidationError`	400	The first field message
Mongoose `CastError`	400	“A supplied identifier is invalid”
Duplicate key (`11000`)	409	“That record already exists”
Anything else	500	Generic message + a `reference` id

21.3 Correlation references and stack traces

```js
const reference = crypto.randomUUID();
logger.error("Unhandled request failure", {
    reference, method: req.method, path: req.path,
    name: error?.name, message: error?.message,
    stack: isProduction() ? undefined : error?.stack
});
return res.status(500).json({ error: "An unexpected server error occurred", code: "INTERNAL_ERROR",
reference });
```

The user sees a random id like `9f3c…`. The full detail is in the server log under the same id. A user can quote it in a support request and a developer can find the exact failure — **without any internal detail being sent to the client**.

> **Security reason — why stack traces are never returned.** A stack trace reveals absolute file paths (`/srv/app/controllers/…`), library versions, function names, and sometimes fragments of data or connection strings. That is a free reconnaissance map. RuWork logs it and returns a generic message **in every environment**, so a misconfigured `NODE_ENV` cannot accidentally expose internals. A test asserts neither the stack nor a Mongo URI appears in the response body.

§ 22

Logging

`utils/logger.js`. There are **no `console.*` calls left in backend runtime code** — a deliberate Phase 10 outcome.

22.1 Two layers of redaction

By key name:

```js
const REDACTED_KEY_PATTERNS = [
    "password","token","secret","authorization","auth","cookie",
    "hash","credential","mongodb_uri","mongodburi","connectionstring","apikey","api_key"
];
```

Substring, case-insensitive — so `password`, `newPassword`, `emailVerificationTokenHash`, and `MONGODB_URI` are all caught.

By value shape (defence in depth):

```js
scrubbed = scrubbed.replace(/\b[A-Za-z0-9_-]{10,}\.[A-Za-z0-9_-]{10,}\.[A-Za-z0-9_-]{10,}\b/g, REDACTED);
// JWT
scrubbed = scrubbed.replace(/mongodb(\+srv)?:\/\/[^\s"']+/gi, REDACTED);
// Mongo URI
scrubbed = scrubbed.replace(/\b[a-f0-9]{64,}\b/gi, REDACTED);
// long hex token
```

> **Why both?** *Key-name matching fails when a secret appears in an unhelpfully named field or embedded in free text — e.g. `message: "auth failed for mongodb://user:pw@host/db"`. Shape matching catches it wherever it hides.*

22.2 Format and silence

Production emits one JSON object per line (machine-parseable for log aggregation); development emits readable text. Output is depth- and size-bounded so one enormous object cannot flood the logs. Under the test gate the logger writes **nothing**, keeping test output clean — asserted by a test.

22.3 Why `console.log` was removed

- It cannot redact — whatever you pass is printed verbatim. The Phase 1 audit found JWTs being printed to logs, which is equivalent to leaking passwords.

- It has no levels, so you cannot filter errors from chatter.
- It has no structure, so aggregators cannot index it.
- It cannot be silenced in tests.

§ 23

Health endpoint

```js
export function getHealth(req, res) {
    const state = CONNECTION_STATES[mongoose.connection.readyState] || "unknown";
    const databaseReady = mongoose.connection.readyState === 1;
    return res.status(databaseReady ? 200 : 503).json({
        status: databaseReady ? "ok" : "degraded",
        uptimeSeconds: Math.round(process.uptime()),
        database: state,
        emailConfigured: isEmailConfigured(),
        time: new Date().toISOString()
    });
}
```

`GET /api/health` is unauthenticated and rate-limit exempt.

`503` when the database is down is the important part. A load balancer polls this endpoint; a `200` means “send me traffic”. An instance that is running but cannot reach MongoDB would fail every request, so it reports `503` and is taken out of rotation automatically.

> **Security reason — why so little detail:** health endpoints are usually public. `emailConfigured` is a boolean, never the SMTP host or user. `database` is one word (`connected` / `disconnected`), never the connection string or host. A test asserts no key contains `uri`, `host`, `password`, `version`, or `secret`. Leaking a database hostname or software version here hands an attacker their first reconnaissance step.

§ 24

Seed/demo script

`scripts/seedDemo.js`, run with `npm run seed:demo`.

Creates one Student, one Provider, two Jobs, two Applications, and one Review, all namespaced by a `ruwork-demo` marker.

Three safety properties:

1. **Refuses to run in production** — `if (isProduction()) throw new Error("Demo seeding is disabled when NODE_ENV is production");`
2. **Requires an explicit `DEMO_PASSWORD`** of at least 8 characters, and never prints it.
3. **Idempotent and namespaced** — it removes only its *own* previous records before recreating them, so it can never touch real accounts, jobs, applications, or audits.

> **Why must it never run in production?** *It would create real, loginable accounts with a known password. If `DEMO_PASSWORD` were guessed or reused, anyone could sign in as an approved Student or Provider.*

⚠ Known defect in this script

`seedDemo.js` creates Applications with **`studentNote`**, but `models/application.js` requires **`applicationNote`**:

```JS
// scripts/seedDemo.js — current (incorrect)
studentNote: "I have prepared research datasets for two previous faculty projects…",
```

Because no MongoDB is configured in this repository, the script has never actually run and Mongoose validation has never rejected it. **Running it today would fail** with a validation error stating `applicationNote` is required.

The fix is to rename the field in both `Application.create` calls. This guide is documentation-only, so the change has **not** been made here — and it should be verified against a real database once applied.

§ 25

Backend testing

25.1 The runner

Node's built-in test runner — no Jest, no Mocha, no extra dependency.

```json
"test": "node --import ./tests/testEnv.js --test"
```

`--import ./tests/testEnv.js` runs that file **before any test module**, which is the only reliable way to set the environment gate before configuration is first read:

```js
process.env.NODE_ENV = "test";
process.env.RUWORK_TEST_MODE = "true";
```

25.2 Test files

FILE	COVERS
`foundation.test.js`	Email normalisation, schema defaults, JWT middleware
`phase2.test.js`	Verification tokens, Admin review APIs, login states, eligibility
`phase4.test.js`	Job schema, pricing, ownership, public query
`phase5.test.js`	Application lifecycle, duplicates, transitions, archiving
`phase6.test.js`	Dashboards, profiles, job history
`phase7.test.js`	Reviews, both aggregates, moderation
`phase8.test.js`	Messages, notifications, participant authorization
`phase9.test.js`	Admin moderation, settings, audits, bounded pagination
`phase10.test.js`	Env validation, logger redaction, error handling, revocation, password lifecycle, health

25.3 Style: no database required

Tests run **without MongoDB**, in about one second, by two techniques.

Testing pure functions directly:

```js
assert.equal(isAllowedStudentEmail("student@sub.ruh.ac.lk"), false);
```

Temporarily replacing model methods (stubbing):

```js
const original = User.findById;
User.findById = async () => account;
try { /* exercise the guard */ } finally { User.findById = original; }
```

The `try/finally` matters — restoring in `finally` means a failing assertion cannot leak a stub into the next test.

A small `response()` helper mimics `res` by recording `statusCode` and `body`, so controllers can be called as plain functions.

25.4 The test gate and production safety

```js
if (useTestFallback(AdminAudit, "create", originalAuditCreate)) return null;
```

where

```js
function useTestFallback(model, method, original) {
    return isTestEnvironment() && mongoose.connection.readyState === 0 && model[method] === original;
}
```

Three conditions must all hold: the explicit test gate is on, there is genuinely no database connection, **and** the method has not been stubbed by a test. `isTestEnvironment()` returns `false` whenever `NODE_ENV=production`, so this shortcut cannot activate in a live deployment — a dropped production connection surfaces as a real failure. A test asserts exactly this.

25.5 Current totals

METRIC	VALUE
Backend tests	**118 / 118 passing**
Backend files passing `node --check`	**57 / 57**
`npm audit`	0 vulnerabilities

§ 26

Backend request walkthroughs

Example A — Student login

`POST /api/users/login` · body `{ email, password }`

STEP	FILE → FUNCTION	WHAT HAPPENS
1	`index.js`	securityHeaders → corsPolicy → express.json → requireObjectBody → apiRateLimiter
2	`routes/userRouter.js`	Matches `POST /login`
3	`middlewears/security.js` → `authRateLimiter`	11th **failed** attempt in 15 min → `429`
4	`controllers/userController.js` → `loginUser()`	Missing field → `400`
5	`utils/account.js` → `normalizeEmail()`	Trim + lowercase
6	`models/user.js` → `User.findOne({ email })`	Indexed lookup
7	`bcrypt.compare()`	Mismatch **or unknown email** → `401` (same message)
8	Eligibility re-checks	role / university / domain → `403 STUDENT_NOT_ELIGIBLE`
9	`isEmailVerified`	→ `403 EMAIL_NOT_VERIFIED`
10	`accountStatus`	→ `403 ACCOUNT_PENDING` / `ACCOUNT_REJECTED`
11	`moderationStatus`	→ `403 ACCOUNT_SUSPENDED`
12	`utils/account.js` → `createAccessToken()`	Signs `{ sub, email, role, tv }`
13	—	`200 { message, token }`

Example B — Student applies to a job

`POST /api/jobs/:jobId/applications` · body `{ applicationNote }`

STEP	FILE → FUNCTION	WHAT HAPPENS
1	`routes/jobRouter.js`	Matches; chain is `authenticateToken → isStudent → requireEligibleRuhunaStudent → applyToJob`
2	`authMiddleware.authenticateToken()`	Verifies signature → `req.user`
3	`authMiddleware.isStudent`	Role must be `student` → else `403`
4	`authMiddleware.requireEligibleRuhunaStudent()`	**Re-reads `User` from MongoDB**; six conditions; `tv` check → `req.studentAccount`
5	`applicationController.applyToJob()`	`normalizedNote()` — 20–1000 chars
6	`models/job.js` → `Job.findOne(...)`	Same five public conditions → `404` if any fails
7	Pricing snapshot	Copies `budgetType` + the matching original price
8	`Application.create(...)`	Identities from `req.studentAccount._id` and the Job
9	Unique index `{ jobId, studentId }`	Duplicate → `11000` → `409 APPLICATION_ALREADY_EXISTS`
10	`utils/communication.js` → `createNotificationSafely()`	`NEW_APPLICATION` to the provider (best-effort)
11	—	`201 { application }`

Example C — Provider accepts an application

`PATCH /api/applications/provider/:id/accept` · body `{ approvedHourlyRate }` or `{ approvedBudget }`

STEP	FILE → FUNCTION	WHAT HAPPENS
1	`routes/applicationRouter.js`	`authenticateToken` (router-level) → `isJobProvider` → `requireApprovedJobProvider`
2	`requireApprovedJobProvider()`	Re-reads provider; approved, verified, not suspended; `tv` valid
3	`acceptApplication()`	Validates the ObjectId → `404`
4	`Application.findById()` + Job lookup	**Ownership verified through the Job**, not the body → `403`
5	`utils/application.js` → `assertApplicationTransition(status,"in_progress","provider")`	Not `pending_review` → `409`
6	`positivePrice()`	Correct approved field for the budget type → else `400`
7	`save()`	Sets `status`, `approved*`, `acceptedAt`; schema hook re-validates
8	`notifyApplication()`	`APPLICATION_ACCEPTED` to the student
9	—	`200 { application }`

Example D — Student sends a message

`POST /api/messages` · body `{ applicationId, content, includeContactNumber? }`

STEP	FILE → FUNCTION	WHAT HAPPENS
1	`routes/messageRouter.js`	`authenticateToken, requireCommunicationParticipant` (router-level)
2	`requireCommunicationParticipant()`	Delegates by role; **Admin → `403`**; sets `req.communicationParticipant`
3	`messageController.sendMessage()`	`assertNoMessageSystemFields()`
4	`authorizedContext()`	Loads the Application; `404` if missing, `403` if not a participant
5	Contact flag checks	Non-boolean → `400`; a Provider setting it → `400`
6	Derive identities	`receiverType/receiverId` computed **from the Application**
7	`messageContent()`	1–2000 chars
8	`sharedContactNumber`	Only if student **and** flag true — read from `participant.account.phoneNumber`
9	`message.save()`	
10	`createNotificationSafely()`	`NEW_MESSAGE` to the receiver (best-effort)
11	—	`201 { item, conversation }`

Example E — Admin hides a job

`PATCH /api/admin/jobs/:id/moderation` · body `{ status: "hidden", reason }`

STEP	FILE → FUNCTION	WHAT HAPPENS
1	`routes/adminRouter.js`	Router-level `authenticateToken, isAdmin, requireAdminAccount`
2	`requireAdminAccount()`	Re-reads the Admin; `tv` valid
3	`adminController.moderateJob()`	`assertOnlyFields(body, ["status","reason"])`
4	Status validation	Must be `visible` or `hidden` → else `400`
5	`Job.findById()`	Invalid/absent → `404`
6	Idempotency	Already hidden → `409`
7	`moderationReason(reason, { required: true })`	5–500 chars on hide
8	`job.save()`	Sets `moderationStatus`, `moderationReason`, `moderatedAt`, `moderatedBy`
9	`createAdminAudit()`	`JOB_HIDDEN`, `adminId` from `req.user.sub`
10	**On audit failure**	Previous values restored, then the error is rethrown
11	—	`200 { job }` — now excluded from public browse, details, and new applications

Example F — Password reset (both halves)

Request — `POST /api/users/password/forgot` · body `{ email }`

STEP	FILE → FUNCTION	WHAT HAPPENS
1	`routes/userRouter.js`	`sensitiveRateLimiter` (20/hour)
2	`passwordController.requestPasswordReset("student")`	Normalises the email
3	`User.findOne(...).select(PRIVATE_RESET_FIELDS)`	Loads the hidden reset fields
4	Guards	Unknown **or** suspended **or** in cooldown → **generic 200**
5	`utils/password.js` → `issueResetToken()`	32 random bytes; stores only SHA-256 + expiry
6	`emailDelivery.sendPasswordResetEmail()`	Emails `{CLIENT_URL}/reset-password?token=…&type=student`
7	On send failure	Token rolled back, **still generic 200**
8	—	Always the same body

Completion — `POST /api/users/password/reset` · body `{ token, newPassword }`

STEP	FILE → FUNCTION	WHAT HAPPENS
1	`routes/userRouter.js`	`authRateLimiter`
2	`resetPassword("student")`	Rejects unexpected fields; validates strength
3	`findAccountByResetToken()`	Format check → hash lookup → **expiry inside the query**
4	Not found / expired / used	`400 RESET_TOKEN_INVALID`
5	`bcrypt.hash()`	Stores the new hash
6	`clearResetToken()`	Makes the link single-use
7	`revokeIssuedTokens()`	`tokenVersion++` — every existing session dies
8	—	`200`, **no token returned** — the user must log in again

§ 27

File-by-file reference

Entry point and configuration

FILE	MAIN RESPONSIBILITY	IMPORTANT FUNCTIONS	USED BY
`index.js`	Build app, order middleware, mount routers, connect DB, listen, shut down	`startServer()`	`npm start`
`package.json`	Dependencies and scripts	`start`, `dev`, `test`, `create-admin`, `seed:demo`	npm
`.env.example`	Documents every variable (no values)	—	Developers

Models

FILE	REPRESENTS	KEY FIELDS	USED BY
`models/user.js`	Student	email, password, accountStatus, moderationStatus, tokenVersion	user/admin/password controllers, guards
`models/jobProvider.js`	Job Provider	companyEmail, accountStatus, moderationStatus, rating summary	provider/admin/password controllers, guards
`models/admin.js`	Admin	email, password, tokenVersion	admin/password controllers, `createAdmin.js`
`models/job.js`	Job posting	jobProviderId, status, pricing, moderation, `providerSuspendedAt`	job/application/admin controllers
`models/application.js`	Student↔Job link	status, pricing snapshot, `applicationNote`	application/review/message controllers
`models/review.js`	Rating + comment	applicationId (unique), rating, moderationStatus	review/admin controllers, aggregates
`models/message.js`	One message	sender/receiver type+id, applicationId, content	message controller
`models/notification.js`	One notification	recipientType/Id, type, isRead	notification controller, `communication.js`
`models/adminAudit.js`	Immutable admin action	adminId, action, entityType, entityId	`utils/admin.js`, admin controller
`models/platformSetting.js`	Singleton settings	three booleans	`utils/admin.js`, registration/job creation

Controllers

FILE	RESPONSIBILITY	IMPORTANT FUNCTIONS	USED BY
controllers/userController.js	Student registration, login, profile, dashboard, history	registerUser, loginUser, getMyProfile, updateMyProfile, getStudentDashboard, getStudentJobHistory	userRouter
controllers/jobProviderController.js	Provider registration, login, profile, dashboard	registerJobProvider, loginJobProvider, updateMyCompanyProfile, getProviderDashboard	jobProviderRouter
controllers/adminController.js	All admin operations	loginAdmin, getAdminDashboard, listRegistrations, approveRegistration, rejectRegistration, moderateStudent, moderateProvider, moderateJob, moderateReview, updateAdminSettings, listAdminAudits	adminRouter
controllers/jobController.js	Job CRUD, browse, public query	createJob, listJobs, getJob, listMyJobs, updateJob, deleteJob, buildPublicJobQuery	jobRouter
controllers/applicationController.js	Application lifecycle	applyToJob, listMyApplications, withdrawMyApplication, cancelMyApplication, acceptApplication, declineApplication, completeApplication	jobRouter, applicationRouter
controllers/reviewController.js	Reviews, public/provider/admin listings	createReview, deleteMyReview, listJobReviews, listProviderReviews, listAdminReviews, deleteReviewAsAdmin	reviewRouter, jobRouter, adminRouter
controllers/messageController.js	Messaging	sendMessage, listConversations, getConversation, getUnreadMessageCount	messageRouter
controllers/notificationController.js	Notifications	listNotifications, markNotificationRead, markAllNotificationsRead, getUnreadNotificationCount	notificationRouter
controllers/emailVerificationController.js	Verify + resend	verifyStudentEmail, resendStudentVerification, verifyJobProviderEmail, resendJobProviderVerification	userRouter, jobProviderRouter
controllers/passwordController.js	Password lifecycle (all roles)	changePassword, requestPasswordReset, resetPassword, logoutAllSessions	all three account routers
controllers/healthController.js	Liveness/readiness	getHealth	index.js

Routes

FILE	PREFIX	NOTES
`routes/userRouter.js`	`/api/users`	Registration, login, verification, password, profile, dashboard, history
`routes/jobProviderRouter.js`	`/api/jobProviders`	Provider equivalents + `/reviews`
`routes/adminRouter.js`	`/api/admin`	`/login` public; **all else** behind three guards
`routes/jobRouter.js`	`/api/jobs`	Public browse/details; provider management; **application creation**; job reviews
`routes/applicationRouter.js`	`/api/applications`	`authenticateToken` router-level; student `/my/*`, provider `/provider/*`
`routes/reviewRouter.js`	`/api/reviews`	Entirely student-only (router-level guards)
`routes/messageRouter.js`	`/api/messages`	Router-level participant guard
`routes/notificationRouter.js`	`/api/notifications`	Router-level participant guard

Middleware

FILE	RESPONSIBILITY	IMPORTANT FUNCTIONS	USED BY
`middlewears/authMiddleware.js`	Authentication, roles, eligibility, revocation	`authenticateToken`, `authorizeRoles`, `isStudent`, `isJobProvider`, `isAdmin`, `requireEligibleRuhunaStudent`, `requireApprovedJobProvider`, `requireAdminAccount`, `requireCommunicationParticipant`	Every router
`middlewears/security.js`	Headers, CORS, rate limits	`securityHeaders`, `corsPolicy`, `apiRateLimiter`, `authRateLimiter`, `sensitiveRateLimiter`	`index.js`, auth routers
`middlewears/errorHandler.js`	Body shape, 404, terminal errors	`requireObjectBody`, `notFoundHandler`, `errorHandler`	`index.js`

Utilities

FILE	RESPONSIBILITY	IMPORTANT FUNCTIONS	USED BY
`utils/account.js`	Roles, email rules, passwords, JWTs	`normalizeEmail`, `isAllowedStudentEmail`, `getPasswordValidationError`, `createAccessToken`, `isTokenVersionCurrent`	Controllers, middleware
`utils/env.js`	Validated configuration	`assertEnvironment`, `getEnvironmentProblems`, `isProduction`, `isTestEnvironment`, `getCorsOrigins`	`index.js`, security, logger, admin
`utils/logger.js`	Redacting structured logs	`logger.info/warn/error`, `redact`	Everywhere that logs
`utils/password.js`	Reset tokens + revocation	`issueResetToken`, `findAccountByResetToken`, `clearResetToken`, `revokeIssuedTokens`	`passwordController`
`utils/emailVerification.js`	Verification tokens + cooldown	`issueVerificationToken`, `findVerificationAccount`, `getVerificationResendWaitSeconds`	Registration, verification
`utils/emailService.js`	SMTP delivery	`emailDelivery.sendVerificationEmail`, `.sendPasswordResetEmail`	Registration, verification, password
`utils/admin.js`	Admin enums, pagination, search, audits, settings	`adminPagination`, `boundedSearch`, `escapeAdminRegex`, `assertOnlyFields`, `createAdminAudit`, `getPlatformSettings`	`adminController`, registration, job creation
`utils/job.js`	Job enums + skill/regex helpers	`JOB_CATEGORIES`, `JOB_STATUSES`, `normalizeSkills`, `escapeRegex`	`models/job.js`, `jobController`
`utils/application.js`	Statuses + transition table	`APPLICATION_STATUSES`, `assertApplicationTransition`, `positivePrice`, `normalizedNote`	`applicationController`, model
`utils/review.js`	Review validation + pagination	`reviewRating`, `reviewComment`, `roundedRating`, `reviewPagination`	`reviewController`, aggregates
`utils/ratingAggregates.js`	Denormalised ratings	`recalculateJobRating`, `recalculateProviderRating`, `recalculateReviewAggregates`	`reviewController`, `adminController`
`utils/communication.js`	Message/notification validation + creation	`messageContent`, `communicationPagination`, `createNotification`, `createNotificationSafely`	Message/notification/application controllers

Scripts and tests

FILE	RESPONSIBILITY
`scripts/createAdmin.js`	Privately provision the first Admin (`npm run create-admin`)
`scripts/seedDemo.js`	Namespaced demo data, non-production only (**has the `studentNote` defect**)
`tests/testEnv.js`	Sets the test gate before any test module loads
`tests/*.test.js`	Nine suites, 118 tests — see §25

§ 28

Common questions

Why separate routes and controllers? Routes answer *"which URL, and who may reach it?"*; controllers answer *"what should happen?"* Keeping them apart means you can see the entire security surface by reading eight short router files, without wading through business logic. It also lets one controller serve several routes — `changePassword("student")` is reused by all three account routers.

Why can't we put everything in `index.js`? It works until it doesn't. You lose the ability to locate a bug quickly, you copy-paste logic that then drifts apart, you cannot unit-test anything in isolation, and every developer edits the same file, so merges conflict constantly.

Why don't we trust the role from the frontend? Because the user controls the frontend completely. They can edit `sessionStorage`, change variables in the debugger, or skip the browser and use curl. Anything the browser *says* is a request, not a fact. The only trustworthy signal is the signed JWT, verified server-side with a secret the browser never sees.

Why use JWT at all? It lets any server instance verify a request using only the secret — no shared session store, no sticky sessions. It is self-contained and scales horizontally.

If the JWT is trustworthy, why re-read the account from MongoDB? Because the token is a snapshot from login time and cannot change. Suspension, rejection, deletion, and password changes all happen *after* it was issued. Without the re-read a suspended user keeps full access until the token expires. See §6.4.

Why is the password hashed? So a database leak does not expose passwords. Hashing is one-way; bcrypt is deliberately slow and salted, making mass cracking impractical. Users reuse passwords, so leaking them harms people far beyond RuWork.

Why is the reset token also hashed? Because it is a temporary credential — whoever holds it can change a password. Storing only the hash means a stolen database dump contains no usable links.

Why do we have `tokenVersion`? JWTs are stateless, so there is no "delete this token". The counter gives a one-integer comparison that invalidates every previously issued token for an account — which is what makes password change and sign-out actually secure.

Why isn't logout just clearing `sessionStorage`? That only removes the *copy* in that browser. The token itself remains valid until it expires, so anyone who captured it could keep using it. Real logout must invalidate the token server-side — hence `POST /logout` incrementing `tokenVersion`. The frontend clears local state **regardless** of whether that call succeeds, so a network failure can never leave someone apparently signed in.

Why use middleware instead of checking inside each controller? Reuse and safety. Written once, applied declaratively per route, impossible to forget. `adminRouter.use(...)` protects all twenty admin endpoints, including ones added next year.

Why `.env`, and why must it never be committed? Secrets must not live in source. Committing `.env` puts your database credentials and JWT secret in git history forever — visible to everyone with repository access, and permanently recoverable even after a later "deletion" commit. It also lets each environment use different values without code changes. Only `.env.example`, which contains names but **no values**, is committed.

Why `async` / `await`? Database and email calls take time. `await` lets you write sequential-looking code that does not block the server: while one request waits on MongoDB, Node serves others. The alternative — nested callbacks — is far harder to read and to error-handle.

Why do we need error-handling middleware? Without it, an unexpected throw either crashes the process or returns a default HTML error page containing a stack trace. Centralising means every failure returns safe, consistent JSON, gets logged with a correlation id, and leaks nothing.

Why rate-limit login? Without it an attacker can try millions of passwords. Ten failures per fifteen minutes makes brute force useless while barely affecting a real person who mistyped.

Why can't an Admin read private messages? Because private work conversations are not the platform's business, and because the smaller the amount of sensitive data an Admin account can reach, the less damage a compromised Admin account does. RuWork reports message *counts* for operational insight instead.

Why don't we hard-delete jobs or accounts? Deleting a job orphans its applications, messages, and reviews, breaking other users' history. Deleting an account makes moderation mistakes unrecoverable and destroys the evidence behind audit records. Reversible status flags achieve the goal — remove it from view — without destroying anything.

Why does hiding a review recalculate ratings? Because `averageRating` is a stored copy, not a live calculation. If the recount were skipped, a hidden abusive review would keep affecting the score forever, and hiding it would achieve nothing. The aggregation excludes `moderationStatus: "hidden"`, so recalculating after every moderation change is what makes hiding meaningful.

Why is `sessionStorage` still considered a limitation? It is readable by any JavaScript running on the page, so a successful XSS attack can steal the token. A `HttpOnly` cookie would be invisible to JavaScript, but that requires CSRF protection and a different architecture — a change too large for Phase 10. `tokenVersion` limits the damage window; it does not eliminate the exposure. It is recorded honestly as a known limitation.

Why must SPA hosts rewrite unknown routes to `index.html`? React Router runs *in the browser*. When a user opens `https://ruwork.lk/reset-password?token=…` directly, the browser asks the **server** for `/reset-password` — a path that does not exist as a file, so a default host returns `404`

and React never loads. Configuring the host to serve `index.html` for unknown paths lets the app boot and route the URL itself. Without it, every emailed verification and reset link is broken.

NEXT: *FRONTEND COMPLETE GUIDE · CODE GLOSSARY · REQUEST & DATA FLOWS*